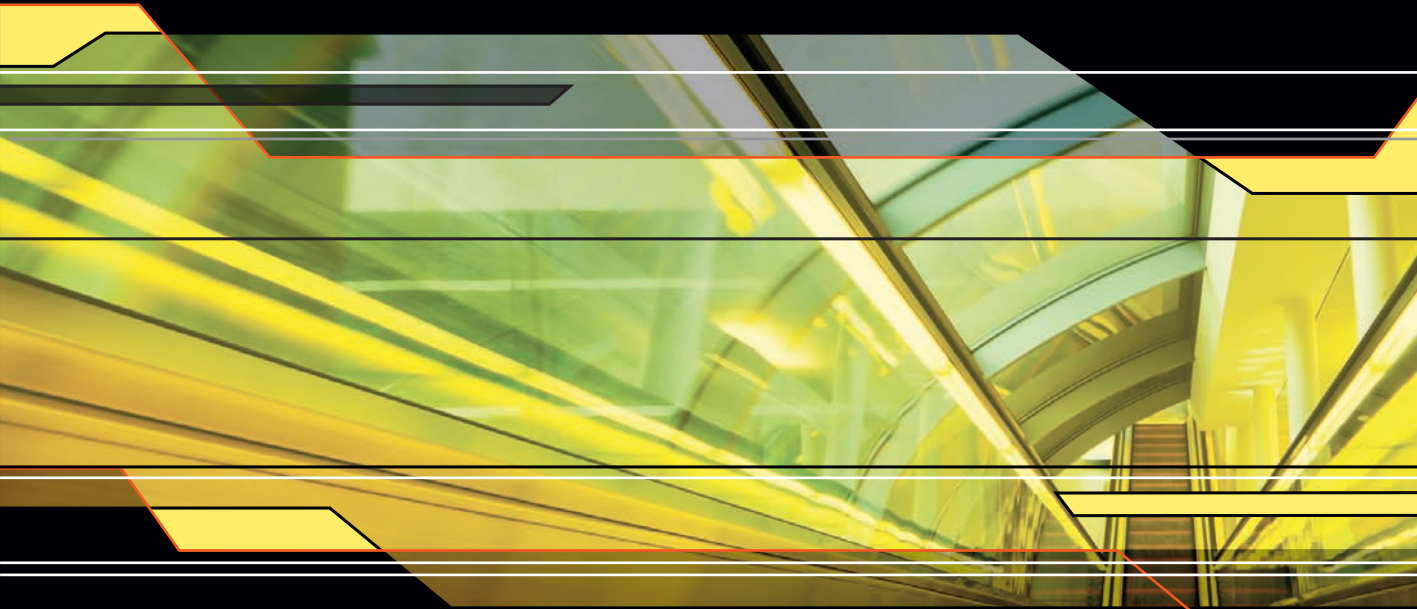


Game Mechanics

Advanced Game Design



Ernest Adams
Joris Dormans

A red line starts at the top left, goes horizontally to the right, then diagonally down to the right, and finally horizontally to the right again.

Game Mechanics

Advanced Game Design



Ernest Adams
Joris Dormans

Game Mechanics: Advanced Game Design

Ernest Adams and Joris Dormans

New Riders Games

1249 Eighth Street
Berkeley, CA 94710
(510) 524-2178
Fax: (510) 524-2221

Find us on the Web at www.newriders.com

To report errors, please send a note to errata@peachpit.com

New Riders Games is an imprint of Peachpit, a division of Pearson Education

Copyright © 2012 Ernest Adams and Joris Dormans

Senior Editor: Karyn Johnson

Developmental Editor: Robyn Thomas

Technical Editor: Tobi Saulnier

Copy Editor: Kim Wimpsett

Production Editor: Cory Borman

Composition: WolfsonDesign

Proofreader: Bethany Stough

Indexer: Valerie Perry

Interior Design: Charlene Will, WolfsonDesign

Cover Design: Peachpit Press/Charlene Will

Notice of Rights

All rights reserved. No part of this book may be reproduced or transmitted in any form by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher. For information on getting permission for reprints and excerpts, contact permissions@peachpit.com. See the next page for image credits.

Notice of Liability

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of the book, neither the authors nor Peachpit shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the instructions contained in this book or by the computer software and hardware products described in it.

Trademarks

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and Peachpit was aware of a trademark claim, the designations appear as requested by the owner of the trademark. All other product names and services identified throughout this book are used in editorial fashion only and for the benefit of such companies with no intention of infringement of the trademark. No such use, or the use of any trade name, is intended to convey endorsement or other affiliation with this book.

ISBN-13: 978-0-321-82027-3

ISBN-10: 0-321-82027-4

9 8 7 6 5 4 3 2 1

Printed and bound in the United States of America

Respectfully dedicated to the memory of Mabel Addis Mergardt,
principal designer of *The Sumerian Game* (later made famous
as *HAMURABI*), the first game with an internal economy that
I ever played.

—Ernest W. Adams

To Marije van Dodeweerd for love.

—Joris Dormans

Acknowledgments

The genesis of this book was a late-night meeting between the two of us during the G-Ameland student game jam festival on a small island off the north coast of the Netherlands. Joris Dormans showed the Machinations framework to Ernest Adams, and Ernest Adams promptly said, “We should write a book about game mechanics.” But it took nearly two years and the advice and assistance of many other people before we were done. Now it is time to thank them.

Our deepest appreciation goes to Mary Ellen Foley and Marije van Dodeweerd, our beloved mates, who patiently tolerated very late nights, missed holidays and weekends, and the occasional rant about the vagaries of the writing process. We’ll make it up to you if we can!

Stéphane Bura suggested that Joris should create an interactive tool when he saw the original, static version of the Machinations diagrams.

Jesper Juul made the invaluable distinction between games of emergence and games of progression that informs the entire book.

Remko Scha had a big impact on the formal scrutiny of the Machinations framework in his capacity as Joris Dormans’s PhD supervisor.

Mary Ellen Foley kindly checked and corrected all our references.

The colleagues and students at the Amsterdam University of Applied Sciences always have been willing test subjects for much of the material that ended up in this book.

We must also thank a number of people for permission to reproduce images: Alexandre Duret-Lutz, for his photo of *The Settlers of Catan*; Andrew Holmes, for his photo of *Kriegsspiel*; Jason Lander, for his photo of *Power Grid*; Johan Bichel Lindegaard, for his photo of *Johan Sebastian Joust*; Wikimedia Commons contributor popperipopp, for his or her photo of the game *Connect Four*. We are also grateful to the Giant Bomb website (www.giantbomb.com), for permission to reproduce screen shots from their collection.

Thanks to Mika Palmu, Philippe Elsass, and all other contributors to *FlashDevelop*, for creating the open source development tool that was used to program the Machinations Tool.

We are extremely grateful to the many anonymous people who have helped to build *Inkscape*, the open source Scalable Vector Graphics editor, without which it would have been much more difficult to produce our illustrations.

As Elrond said, the last place is the place of honor. We thank Margot Hutchison, Ernest Adams's agent, for assistance with the contract. Tobi Saulnier was our wise and sharp-eyed technical editor. Her suggestions are present but invisible throughout the book, and we're deeply grateful that the CEO of a game company would be willing to take the time to help us. Robyn G. Thomas, our tireless (and seemingly sleepless) development editor, pleaded, cajoled, threatened, and oversaw the whole process with her usual flair and attention to detail. And finally, special thanks to Karyn Johnson, senior editor at Peachpit Press, for having the faith in us to let us write the book in the first place.

We hasten to add that the blame for any errors or omissions belongs entirely to us and not to any of the foregoing.

We welcome all comments, questions, and criticism; please write to Joris Dormans at jd@jorisdormans.nl and to Ernest W. Adams at ewadams@designersnotebook.com.

About the Authors

Ernest W. Adams is an American game design consultant and teacher residing in England. In addition to his consulting work, he gives game design workshops and is a popular speaker at conferences and on college campuses. Mr. Adams has worked in the interactive entertainment industry since 1989 and founded the International Game Developers' Association in 1994. He was most recently employed as a lead designer at Bullfrog Productions, and for several years before that, he was the audio/video producer on the *Madden NFL* line of football games at Electronic Arts. In his early career, he was a software engineer, and he has developed online, computer, and console games for machines from the IBM 360 mainframe to the present day. Mr. Adams is the author of four other books, including *Fundamentals of Game Design*, the companion volume to this book. He also writes the Designer's Notebook series of columns on the *Gamasutra* game developers' webzine. His professional website is at www.designersnotebook.com.

Joris Dormans (PhD) is a Dutch lecturer, researcher, and gameplay engineer based in Amsterdam, the Netherlands, working in industry and higher education since 2005. For the past four years, he has been researching formal tools and methods to design game mechanics. His other area of research focuses on how to leverage formal design methods to generate games procedurally. Dr. Dormans has presented papers and hosted workshops on game design on many academic and industry conferences. As an independent freelance game designer, he published and worked on several video and board games. Among these are story-driven adventure games, physical platform games, and a satirical political card game. He has also participated in all Global Game Jams to date. His professional website is at www.jorisdormans.nl.

About the Technical Editor

Tobi Saulnier is founder and CEO of 1st Playable Productions, a game development studio that specializes in design and development of games tailored to specific audiences. Games developed by 1st Playable span numerous genres to appeal to play styles and preferences of each group and include games for young children, girls, middle schoolers, young adults, and some that appeal to broad audiences. The studio also creates games for education. Before joining the game industry in 2000, Tobi managed R&D in embedded and distributed systems at General Electric Research and Development, where she also led initiatives in new product development, software quality, business strategy, and outsourcing. She earned her BS, MS, and PhD in Electrical Engineering from Rensselaer Polytechnic Institute.

Contents

Introduction xi

Who Is This Book For?xii

How Is This Book Organized?xii

Companion Website xiii

CHAPTER 1

Designing Game Mechanics1

Rules Define Games1

Discrete Mechanics vs. Continuous Mechanics9

Mechanics and the Game Design Process12

Prototyping Techniques15

Summary21

Exercises22

CHAPTER 2

Emergence and Progression.....23

The History of Emergence and Progression23

Comparing Emergence and Progression24

Games of Emergence26

Games of Progression30

Structural Differences37

Emergence and Progression Integration39

Summary41

Exercise42

CHAPTER 3

Complex Systems and the Structure of Emergence 43

Gameplay as an Emergent Property of Games43

Structural Qualities of Complex Systems47

Harnessing Emergence in Games57

Summary58

Exercises58

CHAPTER 4

Internal Economy.....59

Elements of Internal Economies	59
Economic Structure	62
Uses for Internal Economies in Games	71
Summary.....	78
Exercises	78

CHAPTER 5

Machinations.....79

The Machinations Framework	79
Machinations Diagram Basic Elements	82
Advanced Node Types	93
Modeling <i>Pac-Man</i>	98
Summary.....	104
Exercises	104

CHAPTER 6

Common Mechanisms107

More Machinations Concepts	107
Feedback Structures in Games	113
Randomness vs. Emergence	126
Example Mechanics	130
Summary.....	144
Exercises	145

CHAPTER 7

Design Patterns147

Introducing Design Patterns	147
Machinations Design Pattern Language	151
Leveraging Patterns for Design	168
Summary.....	169
Exercises	170

CHAPTER 8

Simulating and Balancing Games.....171

Simulated Play Tests	171
Playing with <i>Monopoly</i>	179
Balancing <i>SimWar</i>	187
From Model to Game	195
Summary.....	195
Exercises	196

CHAPTER 9

Building Economies..... 197

Economy-Building Games	197
Analyzing <i>Caesar III</i>	199
Designing <i>Lunar Colony</i>	206
Summary.....	219
Exercises	220

CHAPTER 10

Integrating Level Design and Mechanics221

From Toys to Playgrounds	221
Missions and Game Spaces.....	229
Learning to Play	238
Summary.....	244
Exercises	246

CHAPTER 11

Progression Mechanisms..... 247

Lock-and-Key Mechanisms	247
Emergent Progression	258
Summary.....	270
Exercises	270

CHAPTER 12

Meaningful Mechanics

271

Serious Games.....

271

Communication Theory.....

276

The Semiotics of Games and Simulations

282

Multiple Layers of Meaning

294

Summary.....

299

Exercises

300

APPENDIX A

Machinations Quick Reference

301

APPENDIX B

Design Pattern Library

303

Static Engine.....

303

Dynamic Engine.....

305

Converter Engine

308

Engine Building

311

Static Friction

314

Dynamic Friction

316

Stopping Mechanism

319

Attrition

321

Escalating Challenge

325

Escalating Complexity

327

Arms Race.....

330

Playing Style Reinforcement

333

Multiple Feedback.....

336

Trade

336

Worker Placement.....

336

Slow Cycle.....

336

APPENDIX C

Getting Started with Machinations.....

337

References.....

338

Index.....

341

Online Appendix B.....

B-1

Online Appendix C.....

C-1

Introduction

This is a book about games at their deepest level. No matter how good a game looks, it won't be fun if its mechanics are boring or unbalanced. Game mechanics create gameplay, and to build a great game, you must understand how this happens.

Game Mechanics will show you how to design, test, and tune the core mechanics of a game—any game, from a huge role-playing game to a casual mobile phone game to a board game. Along the way, we'll use many examples from real games that you may know: *Pac-Man*, *Monopoly*, *Civilization*, *StarCraft II*, and others.

This book isn't about building Unreal mods or cloning somebody else's app that's trending right now. It's called *Advanced Game Design* for a reason. We wrote *Game Mechanics* to teach you the timeless principles and practice of mechanics design and, above all, to give you the tools to help you do it—for a class, for a career, for a lifetime.

We also provide you with two unique features that you won't find in any other textbook on game design. One is a new tool called *Machinations* that you can use to visualize and simulate game mechanics on your own computer, without writing any code or using a spreadsheet. *Machinations* allows you to actually *see* what's going on inside your mechanics as they run and to collect statistical data. Not sure if your internal economy is balanced correctly? *Machinations* will let you perform 1,000 runs in a few seconds to see what happens and put all the data at your fingertips. *Machinations* was created by Joris Dormans and is easy to use on any computer that has Adobe Flash Player installed in its web browser. You don't have to use the *Machinations* Tool to benefit from the book, though. It's simply there to help reinforce the concepts.

The other unique feature of *Game Mechanics* is our *design pattern library*. Other authors have tried to document game design patterns before, but ours is the first to distill mechanics design to its essence: the deep structures of game economies that generate challenge and the many kinds of feedback loops. We have assembled a collection of classic patterns in various categories: engines of growth, friction, and escalation, plus additional mechanisms that create stability cycles, arms races, trading systems, and many more. We've made these general enough that you can apply them to any game you build, yet they're practical enough that you can load them in the *Machinations* Tool and see how they work.

Game mechanics lie at the heart of all game design. They implement the living world of the game; they generate active challenges for players to solve in the game world, and they determine the effects of the players' actions on that world. It is the game designer's job to craft mechanics that generate challenging, enjoyable, and well-balanced gameplay.

We wrote this book to help you do that.

Who Is This Book For?

Game Mechanics is aimed at game design students and industry professionals who want to improve their understanding of how to design, build, and test the mechanics of a game. Although we have tried to be as clear as we can, it is not an introductory work. Our book expands on the ideas in another book by Ernest Adams called *Fundamentals of Game Design* (New Riders). We refer to it from time to time, and if you lack a grounding in the basics of game design, you might find it helpful to read the current edition of *Fundamentals of Game Design* first.

The chapters in *Game Mechanics* end with exercises that let you practice the principles we teach. Unlike the exercises in *Fundamentals of Game Design*, many of them require a computer to complete.

How Is This Book Organized?

Game Mechanics is divided into 12 chapters and 2 appendixes that contain valuable reference information. There is also a quick reference guide to Machinations in Appendix A.

Chapter 1, “Designing Game Mechanics,” establishes key ideas and defines terms that we use in the book, and it discusses when and how to go about designing game mechanics. It also lists several forms of prototyping.

Chapter 2, “Emergence and Progression,” introduces and contrasts the important concepts of emergence and progression.

Chapter 3, “Complex Systems and the Structure of Emergence,” describes the nature of complexity and explains how complexity creates emergent, unpredictable game systems.

Chapter 4, “Internal Economy,” offers an overview of internal economies. We show how the structure of an economy creates a game *shape* and produces different phases of gameplay.

Chapter 5, “Machinations,” introduces the Machinations visual design language and the Machinations Tool for building and simulating mechanics. It includes an extensive example using *Pac-Man* as a model.

Chapter 6, “Common Mechanisms,” describes a few of the more advanced features of Machinations and shows how to use it to build and simulate a wide variety of common mechanisms, with examples from many popular game genres.

Chapter 7, “Design Patterns,” provides an overview of the design patterns in our design pattern library and offers suggestions about how to use them to brainstorm new ideas for your designs.

Chapter 8, “Simulating and Balancing Games,” explains how to use Machinations to simulate and balance games, with case studies from *Monopoly* and Will Wright’s *SimWar*.

Chapter 9, “Building Economies,” explores economy-building games, using *Caesar III* as an example, and takes you through the design and refinement process for a new game of our own, *Lunar Colony*.

Chapter 10, “Integrating Level Design and Mechanics,” moves into new territory, looking at how game mechanics integrate with level design and how properly sequenced challenges help the player learn to play.

Chapter 11, “Progression Mechanisms,” discusses two kinds of progression. We start with traditional lock-and-key mechanics and then consider emergent progression systems in which progress is treated a resource within the economy of the game.

Chapter 12, “Meaningful Mechanics,” concludes the book with an exploration of the role of mechanics in transmitting meaning in games that have a real-world message to send. This topic is increasingly important now that game developers are making more *serious games*: games for health care, education, charity, and other purposes.

Appendix A, “Machinations Quick Reference,” lists the most commonly used elements of the Machinations Tool.

Appendix B, “Design Pattern Library,” contains several patterns from our design pattern library. You can find the completed design pattern library in the online Appendix B at www.peachpit.com/gamemechanics and a much more extensive discussion of each design pattern in Chapter 7.

Appendix C, “Getting Started with Machinations,” is available online at www.peachpit.com/gamemechanics and provides a tutorial for using the Machinations Tool.

Companion Website

At www.peachpit.com/gamemechanics you’ll find material for instructors, digital copies of many of the Machinations diagrams used in this book, more design patterns, and a step-by-step tutorial to get you started with Machinations. To get access to this bonus material, all you need to do is register yourself as a Peachpit reader. The material on the website may be updated from time to time, so make sure you have the latest versions.

This page intentionally left blank

CHAPTER 4

Internal Economy

In Chapter 1, we listed five types of mechanics that you might find in a game: physics, internal economy, progression mechanisms, tactical maneuvering, and social interaction. In this chapter, we'll focus on the internal economy.

In real life, an *economy* is a system in which resources are produced, consumed, and exchanged in quantifiable amounts. Many games also include an economy, consisting of the resources the game manipulates and the rules about how they are produced and consumed. However, in games, the internal economy can include all sorts of resources that are not part of a real-life economy. In games, things like health, experience, and skill can be part of the economy just as easily as money, goods, and services. You might not have money in *Doom*, but you do have weapons, ammunition, health, and armor points. In the board game *Risk*, your armies are a vital resource that you must use and risk in a gambit to conquer countries. In *Mario Galaxy*, you collect stars and power-ups to gain extra lives and to get ahead in the game. Almost all genres of games have an internal economy (see Table 1.1 in Chapter 1 for some more examples), even if it does not resemble a real-world economy.

To understand a game's gameplay, it is essential to understand its economy. The economies of some games are small and simple, but no matter how big or small the economy is, creating it is an important design task. It is also one of the few tasks that belongs exclusively to the designer and no one else. To get game physics right, you need to work closely with the programmers; to get a level right, you need to work closely with the story writers and level designers; but you must design the economy on your own. This is the core of the game designer's trade: You craft mechanics to create a game system that is fun and challenging to interact with.

In *Fundamentals of Game Design*, Ernest Adams discussed the internal economy of games. The discussion in this book repeats some of those points and expands the notion of internal economy.

Elements of Internal Economies

In this section, we briefly introduce the basic elements of game economies: *resources*, *entities*, and the four mechanics that allow the resources to be produced, exchanged, and consumed. This is only a summary; if you need a more in-depth introduction, please see Chapter 10, "Core Mechanics," in *Fundamentals of Game Design*.



NOTE We use a very broad definition of the word *economy*. It's not just about money! In an information economy, there are data producers, data processors, and data consumers. Political economy studies the way that political forces influence government policies. Economies about money are called market economies. But we use the term in a more abstract way to refer to any kind of system in which resources—of any type—can be produced, exchanged, and consumed.

Resources

All economies revolve around the flow of resources. Resources refer to any concept that can be measured numerically. Almost anything in a game can function as a resource: money, energy, time, or units under the player's control all are examples of resources, as are items, power-ups, and enemies that oppose the player. Anything the player can produce, gather, collect, or destroy is probably a resource of some sort, but not all resources are under the player's control. Time is a resource that normally disappears by itself, and the player usually cannot change that. Speed is also a resource, although it is generally used as part of a physics engine rather than part of an internal economy. However, not everything in a game is a resource: platforms, walls, and any other type of inactive or fixed-level features are not resources.

Resources can be tangible or intangible. *Tangible resources* have physical properties in the game world. They exist in a particular location and often have to be moved somewhere else. Examples include items the avatar carries around in an inventory or trees that can be harvested in *Warcraft*. In a strategy game, the player's units are also tangible resources that must be directed through the world.

Intangible resources have no physical properties in the game world—they do not occupy space or exist in a particular location. For example, once the trees in *Warcraft* have been harvested, they are changed into lumber, which is intangible. Lumber is just a number—it doesn't exist in a location. The player doesn't need to physically direct lumber to a site to build a new building. Simply having the right amount of lumber is enough to start building, even if the building is constructed far away from the location where the lumber was harvested. *Warcraft's* handling of trees and lumber is a good example of how games can switch between tangible and intangible treatments of resources. Medical kits (tangible) and health points (intangible) in shooter games are another example.

Sometimes it is useful to identify *resources* as either *abstract* or *concrete*. Abstract resources do not really exist in the game but are computed from the current state of the game. For example, in chess you might sacrifice a piece to gain a strategic advantage over your opponent. In this case, "strategic advantage" can be treated as an abstract resource. (Abstract resources are intangible too—obviously, "strategic advantage" is not a thing stored in a location.) Similarly, the altitude of your avatar or units can be advantageous in a platform or strategy game; in this case, it might make sense to treat altitude as a resource, if only as a way of factoring it into the equation for the strategic value of capturing particular positions. The game normally does not explicitly tell the player about abstract resources; they are used only for internal computation.

Note that in video games some resources that might appear to be abstract are in fact quite concrete. For example, *experience points* are not an abstract resource in a role-playing game. Instead, they are an intangible, but real, commodity that must be earned and (sometimes) spent like money. *Happiness* and *reputation* are two more resources used by many games that, although they are intangible, are nevertheless concrete parts of the game.

To design a game's internal economy or to study the internal economy of an existing game, it is most useful to start identifying the main resources and only then describe the mechanisms that govern the relationships between them and how they are produced or consumed.

Entities

Specific quantities of a resource are stored in *entities*. (If you are a programmer, an entity is essentially a variable.) A resource is a general concept, but an entity stores a specific amount of a resource. An entity named "Timer," for example, stores the resource *time*—probably the number of seconds remaining before the end of the game. In *Monopoly*, each player has an entity that stores available cash resources. As the player buys and sells, pays rent and fines, and so on, the amount of cash in the entity changes. When a player pays rent to another player, cash flows from the first player's entity to the second player's entity.

Entities that store one value are called *simple entities*. *Compound entities* are groups of related simple entities, so a compound entity can contain more than one value. For example, a unit in a strategy game normally includes many simple entities that describe its health, damage capability, maximum speed, and so on. Collectively, these make up a compound entity, and the simple entities that make it up are known as its *attributes*. Thus, a unit's health is an attribute of the unit.

Four Economic Functions

Economies commonly include four functions that affect resources and move them around. These are mechanics called *sources*, *drains*, *converters*, and *traders*. We describe them here. Again, this is a summary; for further details, see Chapter 10 of *Fundamentals of Game Design*.

■ **Sources** are mechanics that create new resources out of nothing. At a certain time, or upon certain conditions, a source will generate a new resource and store it in an entity somewhere. Sources may be triggered by events in the game, or they may operate continuously, producing resources at a certain *production rate*. They may also be switched on and off. In simulation games, money is often generated by a source at intervals, with the amount of money created proportional to the population. As another example, some games that involve combat automatically regenerate health over time.

- **Drains** are the opposite of sources: They take resources out of the game, reducing the amount stored in an entity and removing them permanently. In simulation games in which it is necessary to feed a population, the food is drained at a rate proportional to the population. It does not go anywhere or turn into anything else; it simply disappears. In shooter games, ammunition is drained by firing weapons.
- **Converters** turn resources of one kind into another. As we mentioned, in *Warcraft*, trees (a tangible resource) turn into lumber (an intangible one) when the trees are harvested. The act of harvesting is a converter mechanic that converts trees into lumber at a specific rate: A given number of trees will produce a given amount of lumber. Many simulation games include technology upgrades that enable players to improve the efficiency of the converter mechanics in the game, causing them to produce more of the new resource from the old one.
- **Traders** are mechanics that move a resource from one entity to another, and another resource back in the opposite direction, according to an exchange rule. If a player buys a shield from a blacksmith for three gold pieces, the trader mechanic transfers the gold from the player's cash entity to the blacksmith's and transfers the shield from the blacksmith's inventory to the player's. Traders are not the same as converters. Nothing is created or destroyed; things are just exchanged.

Economic Structure

It is not particularly difficult to identify the entities and the resources that comprise an economy, but it is harder to get a good perspective on the system as a whole. If you were to make graphs of the elements in your economy, what shapes would the graphs reveal? Is the amount of a given resource increasing over time? How does the distribution of resources change? Do resources tend to accumulate in the hands of a particular player, or does the system tend to spread them out? Understanding the structure of your economy will help you find the answers.

Economic Shapes

In the real world, people represent features of an economy with charts and figures (Figure 4.1). These graphs have a few interesting properties. At the small scale, their lines move chaotically, but at larger scales, patterns become visible. It is easy to see whether a line is going up or down in the long run and to identify good and bad periods. In other words, we can recognize and identify distinctive shapes and patterns from these types of charts.

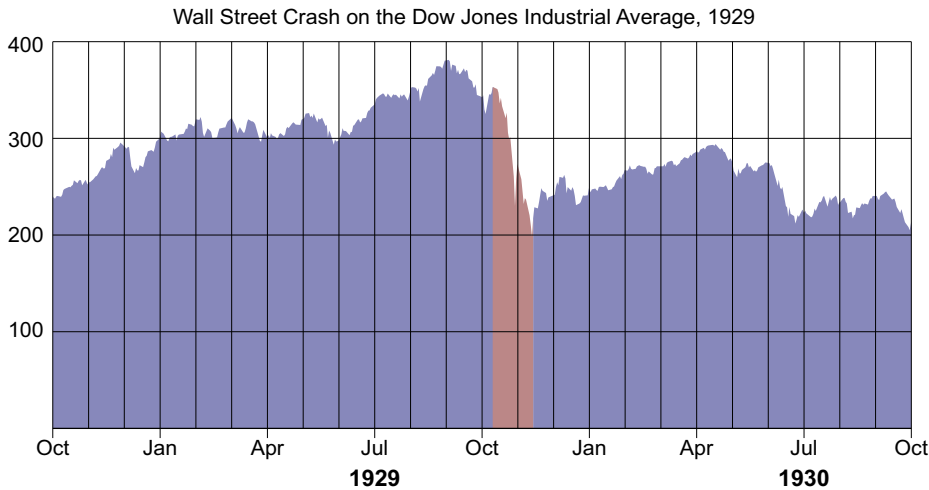


FIGURE 4.1
Graph of the stock market crash leading to the Great Depression. Most movement is chaotic, but the crash is clearly visible.

We can draw similar charts displaying the fortunes of players in a game. As you will see, distinctive shapes and patterns emerge from the internal economy of a game. However, there is no one shape that identifies quality gameplay. What constitutes good gameplay depends on the goals you set for your game and the context that surrounds it. For example, in one game you might want the player to struggle for a long time before managing to come out on top (Figure 4.2). In another, you might aim for quick reversals in fortune and a much shorter play-through (Figure 4.3).

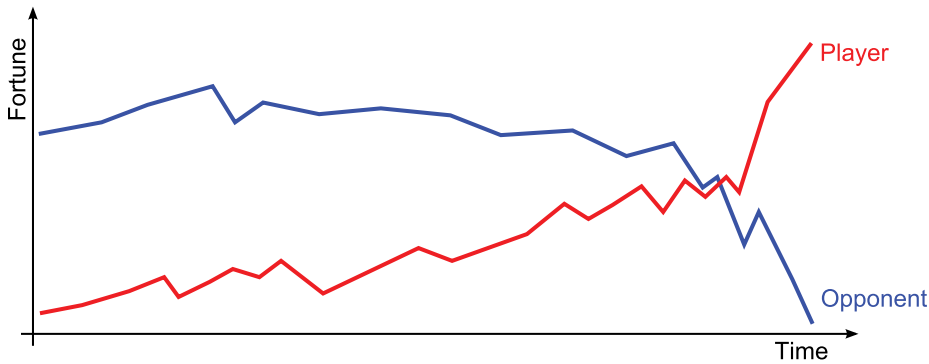
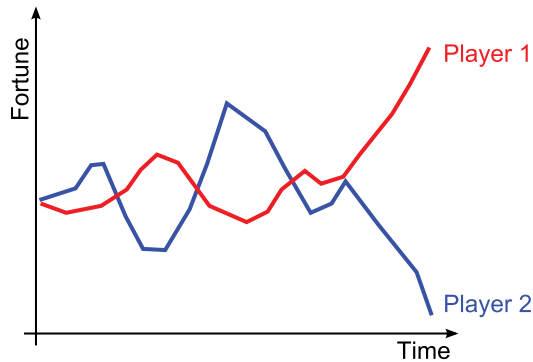


FIGURE 4.2
A long game in which the player triumphs after an extended struggle against a powerful opponent

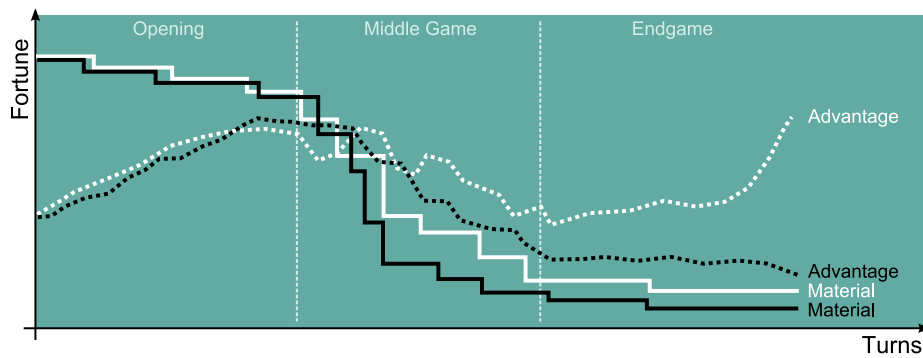
FIGURE 4.3
A short game with quick reversals of fortune



The Shape of a Game of Chess

We can take the development of players' fortunes in a game of chess as a basis for studying shapes in game economies. In chess, the important resources are the players' pieces. Chess players (and computer chess programs) assign a point value to each piece depending on what kind it is. For example, in one system, pawns are worth one point, rooks five, and the queen nine. Adding up the value of all the pieces one player has on the board produces a number called *material*. Players use their pieces to maneuver on the board to gain strategic positions. *Strategic advantage* can be measured as an abstract resource in the game. **Figure 4.4** depicts what might be the course of play between two players in a game of chess.

FIGURE 4.4
The course of a particular game of chess. The color of a line indicates the color of the player it refers to.



You can discover a few important patterns in this chart. To start with, the long-term trend of both players' main resource (material) is downward. As play progresses, players will lose and sacrifice pieces. Gaining material is very difficult. In chess, the only way to gain a piece is to bring a pawn to the other side of the board to be promoted to another, stronger piece, which would lead to an increase of material. This is a rare event that usually initiates a dramatic change of fortune for the players. If we consider only the material, chess appears to be a battle of attrition: Players who can make their material last longest will probably come out on top.

Strategic advantage is more dynamic in the game; it is gained and lost over the course of play. Players use their material to gain strategic advantage or reduce the strategic advantage of their opponents. There is an indirect relationship between the different amounts of material the players have and their ability to gain strategic advantage: If a player has more material, then gaining strategic advantage becomes easier. In turn, strategic advantage might be leveraged to take more pieces of an opponent and reduce that player's material. Sometimes it is possible to sacrifice one of your pieces to gain strategic advantage or to lure your opponent into losing strategic advantage.

A game of chess generally progresses through three different stages: the *opening*, the *middle game*, and the *endgame*. Each stage plays a particular role in the game and is analyzed differently. The opening usually consists of a sequence of prepared and well-studied moves. During the opening, players try to maneuver themselves into a position of advantage. The endgame starts when there are relatively few pieces left, and it becomes safer to involve the king in the game. The middle game falls somewhere between the opening and the endgame, but the boundaries between the stages are not clear. These three stages can also be identified from the economic analysis in Figure 4.4. During the opening, the number of pieces decreases only slowly, while both players build up strategic advantage. The middle game starts when players are exploiting their strategic advantage to take their opponents' pieces; it is characterized by a sharper decline of material. During the endgame, the material stabilizes again as the players focus on their final attempts to push the strategic advantage to a win.



NOTE This analysis of chess is a high-level abstraction to illustrate an economic principle using a familiar game. Classic texts on the theory of chess do not treat it in economic terms, because chess is about checkmating the king, not taking the most pieces. However, our illustration shows that gameplay and game progress can be understood in economic terms even if the game itself is not about economy.

From Mechanics to Shapes

To produce a particular economic shape, you need to know what type of mechanical structures create what shapes. Fortunately, there is a direct relationship between shapes in a game's economy and the structure of its mechanics. In the next sections, we discuss and illustrate the most important building blocks of economic shapes.

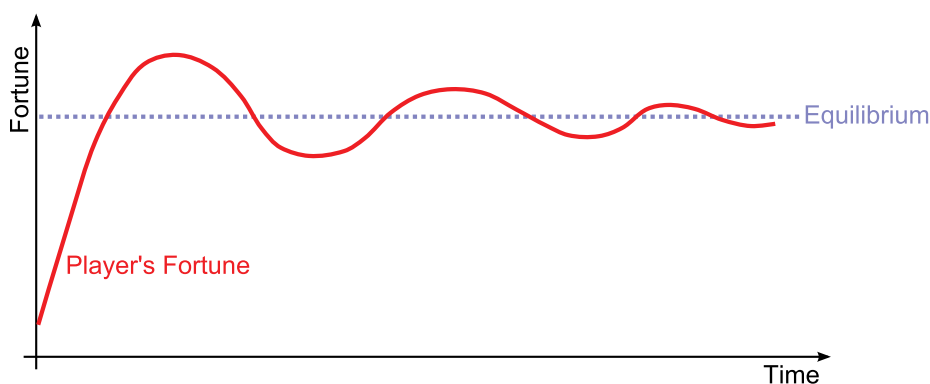
NEGATIVE FEEDBACK CREATES AN EQUILIBRIUM

Negative feedback (as discussed in Chapter 3, "Complex Systems and the Structure of Emergence") is used to create stability in dynamic systems. Negative feedback makes a system resistant to changes: The temperature of your refrigerator is kept constant

even if the temperature outside the refrigerator changes. The point at which the system stabilizes is called the *equilibrium*. **Figure 4.5** displays the effects of negative feedback.

FIGURE 4.5

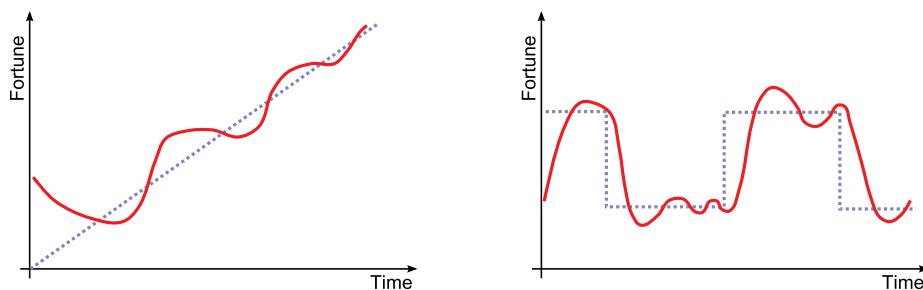
The effect of negative feedback



The simplest shape of the equilibrium is a straight horizontal line, but some systems might have different equilibriums. An equilibrium might change steadily over time or be periodical (**Figure 4.6**). Changing equilibriums requires a dynamic factor that changes more or less independently of the negative feedback mechanism. The outside temperature throughout the year is an example of a periodical equilibrium that is caused by the periodic waxing and waning of the available hours of daylight and the relative strength of the sun.

FIGURE 4.6

Negative feedback on changing equilibriums. On the left, a rising equilibrium; on the right, a periodically changing equilibrium.



POSITIVE FEEDBACK CREATES AN ARMS RACE

Positive feedback creates an exponential curve (**Figure 4.7**). Collecting interest on your savings account is a classic example of such a curve. If the interest is the only source of money going into your savings account, the money will spiral upward, gaining speed as the accumulated sum creates more and more interest over time. In games, this type of positive feedback is often used to create an arms race between multiple players. A good example is the harvesting of raw materials in *StarCraft* (or similar constructions in many other RTS games). In *StarCraft*, you can spend 50

minerals to build a mining unit (called an SCV, for Space Construction Vehicle) that can be used to collect new minerals. If *StarCraft* players set aside a certain portion of their mineral income to build new SCVs, they get the same curve as money in a savings account.

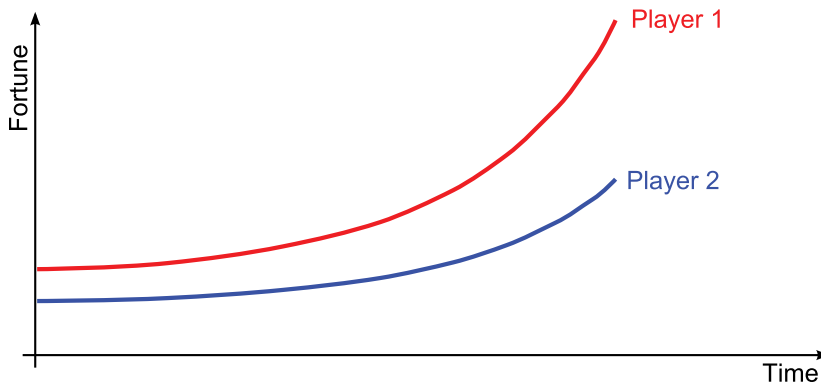


FIGURE 4.7
Positive feedback
creates exponential
curves.

Obviously, *StarCraft* players do not spend their resources only on SCV units. They also need to spend resources to build military units, to expand their bases, and to develop new technology. However, the economic growth potential of a base in *StarCraft* is vital in the long run. Many players build up their defenses first and harvest many resources before pushing to destroy their enemy with a superior capacity to produce military units.

DEADLOCKS AND MUTUAL DEPENDENCIES

Positive feedback mechanisms can create deadlocks and mutual dependencies. In *StarCraft*, to get minerals, you need SCV units, and to get SCV units, you need minerals. These two resources are mutually dependent, and this dependency can lead to a deadlock situation: If you are left without minerals and SCV units, you can never get production started. In fact, you need enough minerals and at least one SCV unit to be able to build a headquarters, a third resource that enables this feedback loop. This deadlock situation is a potential threat. An enemy player might destroy all your SCV units. If this happens when you have spent all your minerals on military units, you are in trouble. It can also be used as a basis for level design. Perhaps you start a mission with military units, some minerals, but no SCV units or headquarters. In this case, you must find and rescue SCV units. Deadlocks and mutual dependencies are characteristics of particular structures in mechanics.

One of the most useful applications of positive feedback in games is that it can be used to make players win quickly once a critical difference is created. As should become clear from Figure 4.7, positive feedback works to amplify small differences: The difference between the balances of two bank accounts with equal interest rates but different initial deposits will only grow over time. This effect of positive feedback can be used to drive a game toward a conclusion after the critical difference has been made. After all, nobody likes to keep playing for long once it has become clear who will win the game.

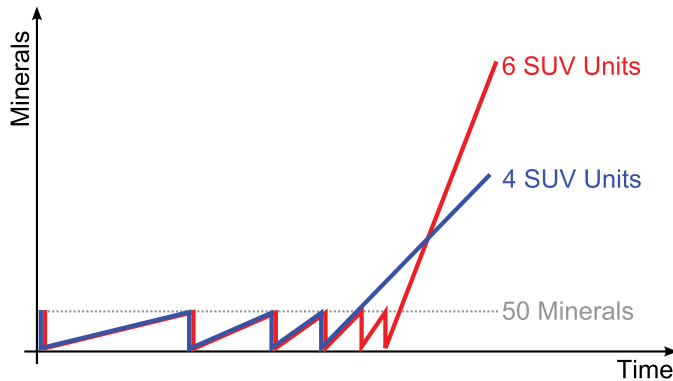
POSITIVE FEEDBACK ON DESTRUCTIVE MECHANISMS

Positive feedback does not always work to make a player win; it can also make a player lose. For example, losing pieces in a game of chess weakens your position and increases the chance that you will lose more pieces; this is the result of a positive feedback loop. Positive feedback can be applied to a destructive mechanism (as is the case with losing material in chess). In this case, it is sometimes called a *downward spiral*. It is important to understand that positive feedback on a destructive mechanism is not the same as negative feedback—negative feedback tends to damp out effects and produce equilibrium. You can also have negative feedback attached to a destructive mechanism. The shooter game *Half-Life* starts spawning more health packs when a player is low on hit points.

LONG-TERM INVESTMENTS VS. SHORT-TERM GAINS

If *StarCraft* were a race to collect as many minerals as possible without any other considerations, would the best strategy be to build a new SCV unit every time you've collected enough minerals? No, not exactly. If you keep spending all your income on new SCVs, you would never save any minerals, which is what you need to win the game. To collect minerals, at some point you need to stop producing SCVs and start stockpiling. The best moment to do this depends on the goals and the constraints of the game—and what the other players do. If the goal is to accumulate the biggest pile of minerals in a limited amount of time or to accumulate a specific number of minerals as quickly as possible, there is an ideal number of SCV units you should produce.

To understand this effect, look at **Figure 4.8**. It shows that as long as you're investing in new SCVs, your minerals do not accumulate. However, as soon as you stop investing, the minerals increase at a steady pace. This pace depends on the number of SCV units you have. The more you have, the faster your minerals will increase. The longer you keep investing, the later you will start accumulating minerals, but you will eventually catch up and overtake anybody who started accumulating before you did. Depending on the target goal, one of those lines is the most effective.

**FIGURE 4.8**

A race of accumulation

It is a good thing *StarCraft* is about more than just collecting minerals. Spending all your minerals on SCV units is a poor strategy because eventually you will be attacked. You have to balance your long-term goals with short-term requirements such as the protection of your base. In addition, some players favor a tactic in which they build up an offensive force quickly in a gambit to overwhelm their opponent before they can build up their defenses—the “tank rush,” which was first made famous in *Command & Conquer: Red Alert*. On some maps, initial access to resources is limited, and you must move around the map quickly to consolidate your access to future resources. Investing in SCV units is a good strategy in the long run, but it requires you take some risk in the beginning, possibly giving up on quick military gains via the tank rush.

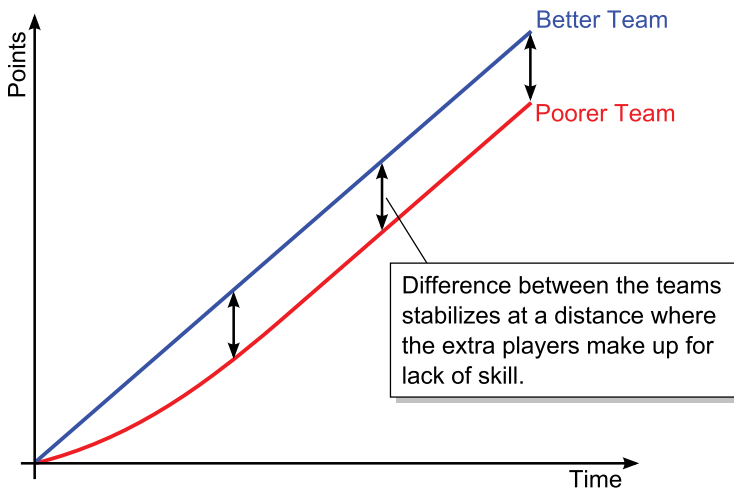
VARIATION FROM PLAYER PERFORMANCE AND RESOURCE DISTRIBUTION

In *StarCraft*, it is not only the number of SCV units that determines the pace at which you harvest minerals. Minerals come from deposits of crystals, which have a particular location on the map. Finding the best location for your base, and micro-managing your SCV units to harvest minerals from crystals effectively, is a skill in itself. These are good examples of how player skill and game world terrain can produce input variation that affects the economic behavior of your game. Of course, the players’ inputs must influence the economy, but it is best if the player’s inputs occur frequently but no one input has too large an effect.

FEEDBACK BASED ON RELATIVE SCORES

During Marc LeBlanc's talk on feedback mechanisms in games at the Game Developers Conference in 1999, he described two alternate versions of basketball. In "negative feedback basketball," for every five points that the leading team is ahead, the trailing team is allowed to field one extra player. In "positive feedback basketball," this effect is reversed: The leading team is allowed to field one extra player for every five points they are ahead. The effects of using the difference between two players to create a feedback mechanism are slightly different from using absolute values to feed this mechanism: The effects of the feedback mechanisms affect the *difference* between the players, not their absolute resources. This can produce some counter-intuitive effects. The economic chart of negative feedback basketball, for example, shows the lead of the better team settling on a stable distance at which the lack of the skill of the trailing team is offset by the extra players they can field (Figure 4.9).

FIGURE 4.9
Score graph of
negative feedback
basketball



DYNAMIC EQUILIBRIUM

The equilibrium that is created by a negative feedback mechanism that is fed by the difference in resources between two players is a dynamic equilibrium: It is not set to a fixed value but is dependent on other, changing factors in the game. You will find that most interesting applications of negative feedback in games are dynamic in this way. Making the equilibrium of a negative feedback loop dynamic by making it dependent on the relative fortune of multiple players, or other factors in the game, is a good way to move away from a too predictable balance created by a nondynamic equilibrium. With experience, knowledge, and skill, you will be able to combine several factors to compose dynamic equilibriums that are periodic, are progressive, or follow another desired shape.

When two teams are playing positive feedback basketball, the differences in skills are aggravated. When one side is better than the other, this will result in a very one-sided match. However, when both sides are closely matched, a different pattern emerges: The game will probably remain close, until one side manages to take a decisive lead after which the match becomes very one-sided again. In this latter case, a small difference in skill, an extra effort, or sheer luck can become the decisive factor.

In Chapter 6, we explore the gameplay effects of positive and negative feedback on basketball in more detail.

RUBBERBANDING IS NEGATIVE FEEDBACK ON RELATIVE POSITION

Racing games frequently use negative feedback based on the players' position in the field to keep the race tight and exciting. This mechanism is often referred to as *rubberbanding*, because it seems to players as if the other cars are attached to theirs by a rubber band—they never get too far ahead or too far behind. Some games implement rubberbanding by simply slowing leading cars down and speeding trailing cars up. Other games use more subtle negative feedback mechanics to reach similar effects. In *MarioKart*, players are awarded with a random power after picking up a power-up. However, trailing players have a better chance of picking up a more powerful power-up than leading ones do. In addition, because most weapon power-ups in *MarioKart* are used on opponents in front of the player, the leader of the field is a target more often than the player in the last position. This causes the lead to change hands frequently and increases the excitement of the game, increasing the likelihood of a last-minute surge past the leader.

Uses for Internal Economies in Games

In the previous sections, we discussed the elements and common structures of internal game economies. In this section, we will discuss how game economies are typically used in games of different genres. Table 1.1 provided a quick overview of some mechanics that are typically part of that economy. Now, we will discuss the typical economic structures found across game genres in more detail.

Use an Internal Economy to Complement Physics

Obviously, physics make up the largest part of action games' core mechanics. Physics are used to test the player's dexterity, timing, and accuracy. Still, most action games add an internal economy to create an integral reward system or to establish a system of power-ups that requires resources. In a way, the simple use of a scoring system adds economic mechanics to many action games. If you collect points for taking out enemies, players will have to consider how much they will invest to take out that

enemy. Will they put their avatars at risk, or will they waste ammunition or some sort of energy that cannot easily be regained?

Super Mario Brothers and many other similar platform games use a simple economy to create a reward system. In *Super Mario Brothers*, you can collect coins to gain extra lives. Because you need to collect quite a few coins, the designer can place them liberally throughout a level and add or remove them during play-testing without affecting the economy significantly. In this way, coins can be used to guide a player through a level. (Collectible objects that are used to guide players are often called *breadcrumbs*.) It is safe to assume that you are able to reach all coins, so if you spot a coin, there must be a way to reach it. This creates the opportunity to reward skillful players for reaching difficult places in the game. Used in this way, the internal economy of the game can be very simple. However, even a simple economy like this already involves a feedback loop. If players go out of their way to collect many coins, they will gain more lives, thus allowing them to take more risks to collect more coins.

When setting up a system like this, you must be careful to balance the risks and rewards. If you lure players into deadly traps with just a single coin, you are inviting them to risk a life to gain a single coin. That simply isn't fair, and the player will probably feel cheated. As a designer, you have a responsibility to match the risks and rewards, especially when they are placed close to the path novice players will take. (Creating a reward that the player can see but *never* reach is even worse—it causes players to take risks for rewards they can never obtain.)

Power-ups, including weapons and ammunition in first-person shooters, create a similar economy. Power-ups and ammo can be rewards in themselves, challenging the player to try to eliminate all enemies in a level. As a game designer, you have to make sure that the balance is right. In some games, it is perfectly all right if killing enemies will, on average, cost more bullets than the players can loot from their remains. However, if this leads to a situation in which the player is eventually short on the proper ammo for the big confrontation with a boss character, you risk penalizing players for making an effort in the game. In survival-oriented first-person shooters, creating a scarce economy of weapons and ammo is generally a good thing because it adds to the tension and the drama, but it is a difficult balance to create. If your shooter is more action-oriented, then it is probably best to make sure there is plenty of ammo for the player, and you should make sure that taking out extra enemies is properly rewarded.

Use an Internal Economy to Influence Progression

The internal economy of a game can also be used to influence progression through a game that involves movement. For example, power-ups and unique weapons can play a special role in an action game's economy. They can be used to gain access to new locations. A double-jump ability in a platform game will allow the player to reach higher platforms that were initially unreachable. In economic terms, you can think of these abilities as new resources to produce the abstract resource *access*.

Access can be used to gain more rewards or can be required to progress through the game.

In both cases, as a designer, you should be wary of a deadlock situation. For example, you might have a special enemy guard the exit of a level. Somewhere in the same level there is a unique weapon that is required to kill that enemy with a single shot. The weapon is usable throughout the level. When the player finds the weapon, it is loaded with ten bullets, and there are no more until the next level—but the player doesn't know this the first time playing. Now, a first-time player finds the weapon, fires a couple of shots to experiment with it, uses it on a couple of other enemies, and finds himself at the exit with one bullet left. The player fires and misses. You have just created a deadlock situation. The player needs access to the next level to gain bullets but needs bullets to gain access.

DEADLOCK RESOLUTION IN ZELDA

In many Zelda games, players frequently must use consumable items—arrows or bombs—to gain access to new areas. This creates a risk of deadlocks, if the player runs out of the items needed. The designers of Zelda games prevent these no-win situations by making sure there are plenty of renewable sources for the required resources. Dungeons are littered with useful pots that yield these resources if the player destroys them (**Figure 4.10**). Broken pots are mysteriously restored as the player moves from room to room, creating a source that is replenished from time to time. Because the pots can contain anything, as a designer you can use a mechanism like this to provide the player with any resource required. You can even use it as a way of providing gameplay hints: If players are finding a lot of arrows, they are probably going to need a bow soon.



FIGURE 4.10 Pottery is a useful source in Zelda games.

Use an Internal Economy to Add Strategic Gameplay

It is surprising how many of the strategic challenges in real-time strategy games are economic in nature. In a typical game of *StarCraft*, you probably spend more time managing the economy than fighting the battle. Including an internal economy is a good way to introduce a strategic dimension to a game that operates on a larger time span than most physical and/or tactical action.

One of the reasons that most real-time strategy games have elaborate internal economies is that these economies allow the games to reward planning and long-term investments. A game about military conflict with little forward planning and no long-term investments would be a game of tactics rather than strategy, because it would probably be more about maneuvering units on the battle field. To sustain a level of strategic interaction, a game's internal economy needs to be more complicated than the internal economies that simply complement the physics of an action game. Economies in strategy games usually involve multiple resources and involve many feedback loops and interrelationships. Setting up an economy like that for the first time is challenging, and finding the right balance is even more difficult. As a designer, you need to understand the elements of the economy and develop a keen sense to judge its dynamic effects. Even if you have years of experience, it is easy to make mistakes: There have been many tweaks to the economy of games like *StarCraft* to retain the right balance after players developed new strategies, even after the game had been long published!

Even without a focus on the economics of production (such as *StarCraft*'s minerals and SCV units), internal economies can add strategic depth to almost any game. In most cases, this involves planning to use the available resources wisely. As already discussed, the economy of chess can be understood in terms of material (playing pieces) and strategic advantage. Chess is not about production, and gaining a piece in chess is unusual. Rather, the game is about using and sometimes sacrificing your material in order to produce as much strategic advantage as possible. In other words, chess is all about getting the most mileage out of your pieces.

You can find something similar in the game *Prince of Persia: The Sands of Time*. In this action-adventure game, the player progresses through many levels filled with dexterity and combat challenges. Early in the game, the player is awarded a magical dagger that allows that player to control time. If anything goes wrong, the player can use sand from the dagger to rewind time and to try again. This power can also be used during combat, for example just after the player has taken a big hit. In addition, the player can use sand as a magical power to freeze time. This helps when battling multiple enemies. The sand is not limitless, however. The player can rewind time only so often, but fortunately, defeating enemies provides the player with new sand. This means that, in addition to the usual action-oriented gameplay, the player has to manage a vital resource. The player must decide when is the best time to invest some sand. Different players will have different ideas about when they should use their sand. Some will use it more often to help out with combat, while

others will prefer to save it for challenging jumping puzzles. In this way, the sand is a versatile resource: Players are able to use it to boost their performance where they need it most.

Use an Internal Economy to Create Large Probability Spaces

As internal economies grow more complex, the probability space of your game expands quickly. Games with a large probability space tend to offer more replay value, because players will have more options to explore than is generally achievable with a single play-through. Another benefit is that these games can also create a more personal experience, because the performance of players and their choices directly affect what parts of the probability space open up for exploration.

Games that use an internal economy to govern character development, technology, growth, or vehicle upgrades often use an internal currency to provide options to the player. This is a typical gameplay feature found in role-playing games, in which players spend in-game money to outfit their characters and spend experience points to develop skills and abilities. It is also found in certain racing games that allow players to tune or upgrade their vehicles between (or sometimes even during) races. As long as there are enough options and the options present really different solutions to problems encountered in the game, or are otherwise important to the player, this is a good strategy.

When using an internal economy to customize the gameplay, there are three things you need to watch out for. First, in an online role-playing game, if a particular combination of items and skills is more efficient than others, players will quickly identify and share this information, and the economy will be thrown off-balance. Either players will choose only that option, effectively reducing the probability space and creating a monotonous experience, or they will complain that they cannot keep up with players who did. In games like this, it is important to understand that customization features are best balanced by some sort of negative feedback. Role-playing games usually implement many negative feedback mechanisms for this reason: Every time characters gain a level and improved skills, they need more experience points to get to the next level. This effectively works to reduce the differences in levels and abilities and requires more investment from a player for each level earned.

Second, you have to be sure that the probability space is large enough that players do not end up exploring it entirely in one play session. For example, if in a role-playing game players have a rating between 1 and 5 for the attributes of strength, dexterity, and wisdom, and the player can choose which one to increase from time to time, it is generally a poor design decision to require them to upgrade all these attributes to the maximum in order to finish the game. Similarly, if the player has only limited choice over what order to upgrade her attributes, the consequences of those choices are reduced. A good way to include choices that have real consequences is to create choices that exclude each other. For example, players can generally choose only one

class for their character in a role-playing game. Each class should have a unique set of different skills and abilities. In *Deus Ex*, the player is also presented with choices to improve the cyborg character that have gameplay consequences: The player might be forced to choose between installing a module that will render the character invisible for short periods and a special type of subdermal armor that will make the character much more resistant to damage.

Third, you should ideally design your levels in such a way that players can use different strategies to complete them. For example, in *Deus Ex*, the player can choose to develop a character in different ways. The player can focus on combat, stealth, or hacking as alternative ways of solving the many challenges in the game. This means that almost every level has multiple solutions. This is not an easy balance to strike. If you estimate that the player has managed to upgrade three options before a certain level, you have to take into account that the player upgraded the combat abilities three times, stealth three times, hacking three times, or perhaps all of them once. In *Deus Ex*, this problem is even more pronounced because all the sources of experience points that you require to upgrade are not renewable: You gain them for progressing and performing certain side quests. Going back to a previous area to harvest some more experience is not an option.

This example illustrates that the levels in games that permit customization must be more flexible, and more general, than in conventional action games, because you don't know exactly what abilities the player's avatar will have. *Deus Ex Human Revolution* contained a flaw: It allowed the players different ways to play the game but only one way to beat the boss characters, which defeated the point of allowing the players to customize their avatars.

Tips for Economy Construction Games

Games in which the player builds an economy, such as construction and management simulations, tend to have large and complex internal economies. *SimCity* is a good example. As players zone areas and build infrastructure, they use these building blocks to craft an economic structure that produces the resources they need to increase it even further. Building a game like this requires the designer to assemble a toolbox of mechanics that the player can combine in many interesting ways. This is even harder than designing a complete, functional, and balanced economy yourself. You have to be aware of all the different ways your economic building blocks combine. When successful, playing the game can be very rewarding, because the economy the players build up through play directly reflects their choices and strategies. This is why no two cities in *SimCity* are alike.

If you are designing an economy construction game, there are three strategies that can help you keep the complexity of your task under control:

- **Don't introduce all the player's building blocks at once.** Construction and management simulations typically allow the player to build something—a farm, factory, or city, for example—out of elementary units, building blocks, that play a role in the economy. (In *SimCity*, these are zoned land and specialized buildings.) It is a good idea to gently introduce players to the different elements in your game, a few at a time. This makes it easier to control the probability space, at least initially. By allowing certain building blocks and disallowing others, you can craft scenarios and create special challenges. If your game has no distinct levels or special scenarios, make sure that not all building options are available from the start. Have players accumulate resources before they can use the more advanced building blocks that unlock new options. *Civilization* is an excellent example of an economy construction game in which most of the building blocks are locked at the beginning of the game and must be unlocked one by one before the players can use them.
- **Be aware of the meta-economic structure.** In an ideal economy construction game, the number of ways of putting the economic building blocks together is endless. However, in most such games, certain approaches are better than others (and in games with a victory condition, some approaches are unwinnable). As a designer, you should be aware of typical constructions that might be called *meta-economic structures*. For example, in *SimCity*, a particular mix of industrial, residential, and commercial zones will prove to be very effective. Players will probably discover these structures quickly and follow them closely. One difficult, but effective, way of dealing with patterns that could become too dominant is to make sure that patterns that are effective early in the game cease to be effective later. For example, a particular layout of zones might be an effective way to grow your population initially but causes a lot of pollution in the long run. Slow-working, destructive positive feedback is a good mechanism to create this sort of effect.
- **Use maps to produce variety and constrain the possibility space.** *SimCity* and *Civilization* wouldn't be nearly as much fun if you could build your city or empire on an ideal piece of land. Part of the challenge of these games is to deal with the limitations of the virtual environment's initial state. As a designer, you can use the design of the map to constrain players or to present opportunities. So, although there might be a best way of building the economy (something that we might call a *dominant* meta-economic structure), it is simply not possible to do so in particular terrain. This forces players to improvise, and rewards players who are more flexible and versatile. In *SimCity*, the disaster scenarios in which players can unleash several natural disasters on their cities challenges their improvisation and flexibility in a similar vein; and of course, *SimCity* also generates disasters at random, setting back the player's progress.

Summary

In this chapter, we introduced the essential elements of an internal economy: resources, entities, and some of the mechanics that manipulate them, including sources, drains, converters, and traders. We examined the concept of economic shapes as seen through graphs and showed how different mechanical structures can produce different shapes. Negative feedback creates equilibrium, while positive feedback creates an arms race among opponents. Implemented another way, positive feedback can produce a downward spiral, because a player finds it harder and harder to grow his economy. Feedback systems based on relationships between two players can produce effects that keep games close or tend to cause the player in the lead to stay in the lead.

Game designers can use internal economics in many ways to make games interesting, enriching both the progression of a game and the strategic choices a player has to make. The internal economy also affects the competitive landscape between diverse or closely matched players in multiplayer games. The chapter ended with specific suggestions about how to build games in which players construct an economy, as in *SimCity*.

Exercises

1. Identify the resources and economic functions in a published game. (Your instructor may specify particular games to study.)
2. Find an example of a game (not referred to in this chapter) that exhibits one of these properties: negative feedback with periodic equilibrium, a downward spiral, a short-term versus long-term investment trade-off, feedback based on players' relative scores, or rubberbanding. Explain which resources are involved, and show how the game's mechanics produce the effect you discovered.
3. Find an example of a game (other than a Zelda game) in which a deadlock may occur. Does the game provide a means of breaking the deadlock? Explain.

INDEX

400 Project, 150

A

abstraction

- elimination, 286–287
- in Machinations diagrams, 81–82
- process of, 286–287
- simplification, 286–287
- in simulations, 286–287

action games

- level progression, 131
- mechanics, 8
- power-ups and collectibles in, 131–133

actions

- challenges associated with, 43–44
- effect of, 43
- unexpected, 44

Adams, Ernest

- definition of games, 1
- Fundamentals of Game Design*, 59
- hierarchy of challenges, 229
- player-centric design, 169, 292

adventure games, mechanics, 8

Alexander, Christopher, 148

America's Army, 287, 299

analogous simulation, 288–289, 291–293

Angry Birds, 31

- physics, 6
- strategy in, 10
- vs. *World of Goo*, 10–11

AP (artificial player). *See* artificial players; players

arms race pattern

- applicability, 330
- collaborations, 331
- consequences, 331
- examples, 332
- implementation, 331–332
- intent, 330
- participants, 331
- related patterns, 332

structure, 330

type, 330

Art of Computer Game Design, 232

artificial players. *See also* direct commands; Machinations diagrams; players

activate(parameter) command, 175

adding to Machinations

diagrams, 172

additive script condition, 174

color-coded, 175

deactivate() command, 175

defining in SimWar, 191–192

designing strategies, 177

diagram with, 171

direct commands, 172–173

endTurn() command, 175

equality script condition, 174

if statements, 173–175

linking, 178

logical and script condition, 174

logical or script condition, 174

in *Monopoly*, 179

multiplicative script condition, 174

nonequality script condition, 174

purpose of, 177

Quick Run option, 176–177

relational script condition, 174

removing randomness, 177–178

script box, 172

script conditions, 173–174

selecting node for, 172

stopDiagram(message)

command, 174

values in conditions, 175

Ashmore and Nietzsche, 35

attrition pattern

applicability, 321

collaborations, 322

consequences, 322

examples, 323–324

implementation, 322–323

intent, 321

motivation, 321

participants, 322

related patterns, 325

structure, 322

type, 321

avatars, customizing attributes of, 135–136

B

basketball

Difference pool, 116

negative feedback, 70, 116–117

positive feedback, 70–71, 116–117

battle, mapping, 141

Beck, John, 272

“Behavioral Game Design,” 109

Bioshock

moral layer, 295

physical layer, 295

political layer, 295

as satire, 295

Björk, Staffan, 151

blackjack game, length of, 2

board games

randomness vs. emergence in, 128

reliance on emergent progression, 259

Bogost, Ian, 287

bombing keys, example of, 254

Boulder Dash, 9, 26

Brathwaite, Brenda, 297

breadcrumbs, defined, 72

C

Caesar III

advantages, 200

buildings, 204–205

city economy, 199

connecting components, 206

connections between elements, 200–201

converter engine, 202

- Caesar III (*continued*)
 - described, 199
 - design patterns, 202
 - dominant economic structure, 202–203
 - dynamic friction, 202
 - economic buildings, 201
 - economic relationships, 200
 - engine building, 202
 - farms, 204
 - as game of emergence, 206
 - landscape, 202–203
 - maps, 203
 - markets, 205
 - mechanisms, 201
 - missions, 203
 - money for building, 203
 - multiple feedback, 202
 - negative feedback, 203–204
 - phases of progression, 206
 - players, 200
 - progress in, 224
 - residences, 204
 - resources, 199
 - restricting players, 202–203
- Caillois, Roger, 222
- cartoon physics, explained, 6
- “The Case for Game Design Patterns,” 150
- Caylus* board game, activators in, 92, 128
- cellular automata
 - Game of Life, 53
 - generation, 48
 - study of, 48
 - threshold for complexity, 50
 - tower defense games, 50
 - Wolfram’s, 48–49
- challenge to adventure, example of, 36
- challenges
 - adding to improve experience, 231–232
 - atomic, 229
 - focusing on, 229
 - relationship to actions, 43–44
- chance, relying on, 126
- chaos vs. order
 - emergent systems, 45–47
 - periodic systems, 45–46
- characters, customizing attributes of, 135–136
- charts, using, 63
- chess game
 - charting patterns, 65
 - endgame, 65
 - long-term trend, 65
 - material number, 64
 - middle game, 65
 - opening stage, 65
 - shape of, 64–65
 - strategic advantage, 64–65
- choice, creating via enemies, 231
- Chomsky, Noam, 293
- Church, Doug, 149–150
- Civilization*, 28–30
 - city economy of, 318
 - development phases in, 47
 - discrete mechanics, 29
 - economies in, 197–198
 - economy construction, 77
 - gameplay phases, 30
 - golden ages, 30
 - historical periods, 30
 - phases, 29–30
 - random maps in, 126
 - reverse triggers in, 111
 - vs. *StarCraft*, 40
 - strategies, 29
 - technology tree, 144
- Civilization V*, negative feedback in, 52
- closed circuits, creating feedback with, 114–115
- cognitive effort vs. speed, 232
- collectibles, indicating, 131–133
- color-coding
 - delays and queues, 113
 - Machinations diagrams, 112–113
- combat construction
 - example of, 141–142
 - in *SimWar*, 189
- Command & Conquer: Red Alert*, 69
- communication
 - interactivity of, 278
 - model of, 276
- communication theory
 - art and entertainment, 277
 - channel, 276
 - design challenges, 280–281
 - functions, 277
- mechanics sending messages, 279–280
- medium and message, 277–279
- message, 276
- poetic function, 277
- receiver, 276
- sender, 276
- signal, 276
- complex systems. *See also* emergence; science of complexity
 - active and interconnected parts, 48–51
 - behavioral patterns, 53–56
 - behaviors, 46
 - categorizing emergence, 56–57
 - cell activity, 50
 - cellular automata, 48
 - defined, 45
 - destabilizing, 51–53
 - dynamic behavior, 49–50
 - ecosystems, 51–53
 - emergence in, 47
 - feedback loops, 51–53
 - intentional emergence, 56
 - long-range communication, 49
 - multiple emergence, 56
 - nominal emergence, 56
 - simple cells, 49
 - simple parts in, 26–27
 - stabilizing, 51–53
 - strong emergence, 57
 - weak emergence, 56
 - weather, 47
- complexity
 - of game behavior, 45
 - of rules, 45
- complexity barrier, explained, 37
- complexity theory, applying to phase transitions, 267
- concept stage, 13
- Connect Four*
 - gravity in, 28
 - vs. tic-tac-toe, 27
- consistency vs. realism, 44
- continuous mechanics, 9
- converter element, elaborations for, 164
- converter engine
 - applicability, 308
 - in *Caesar III*, 202

- collaborations, 308
- consequences, 309
- examples, 309–311
- implementation, 309
- intent, 308
- motivation, 308
- participants, 308
- related patterns, 311
- structure, 308
- type, 308
- converters
 - explained, 62
 - vs. traders, 97
 - using with resources, 96
- Conway, John, 53, 56
- Cook, Daniel, 238–239
- Copenhagen Games Collective, 5
- core mechanics
 - explained, 4
 - of video games, 4
- Counter-Strike*, gun fights in, 24
- Crash Bandicoot*
 - Kata stage, 242
 - Kihon stage, 241–244
 - Kihon-kata stage, 241
 - Kumite stage, 242
- Crawford, Chris, 25, 232

D

- data intensity, 25
- deadlocks
 - being aware of, 73
 - resolution in *Zelda* games, 73
- delays, using in *Machinations* diagrams, 110–111
- Descent: Journeys in the Dark*, 305
- design, player-centric, 169
- design patterns. *See also* pattern descriptions; pattern language
 - arms race, 158
 - attrition, 156
 - brainstorming with, 168–169
 - in *Caesar III*, 202
 - combining, 161
 - converter engine, 154, 202, 216
 - defined, 148
 - vs. design vocabularies, 149
 - dynamic engine, 153, 162, 188, 212
 - dynamic friction, 155, 186, 202, 255–256
 - engine building, 154, 212
 - Engines category, 153–154
 - escalating challenge, 157, 232
 - escalating complexity, 157, 232, 269
 - Escalation category, 157–158
 - Friction category, 155–156
 - in games, 151
 - history of, 148–149
 - improving, 168
 - law of diminishing returns, 156
 - multiple feedback, 158
 - playing style reinforcement, 158
 - slow cycle, 160–161
 - static engine, 153
 - static friction, 155, 268
 - stopping mechanism, 156, 269
 - trade, 159
 - worker placement, 160
- design process. *See* game design process
- design tools
 - investing in, 166–167
 - support for creativity, 167
- design vocabularies, 149
 - intention, 150
 - online, 150
 - perceivable consequence, 150
 - story, 150
- determinability. *See also* feedback structures
 - deterministic, 124
 - multiplayer-dynamic, 124
 - player skill, 124
 - random flow rates, 124
 - strategy, 124
 - Tetris* example, 125
- deterministic behavior, symbols for, 125
- deterministic harvesting game, 129–130
- deterministic processes, explained, 2
- Deus Ex*, 25, 76
- Diablo*-style inventory, 289
- dice, rolling, 290
- die symbol, appearance of, 84
- Diplomacy* board game, unpredictability of, 3
- direct commands. *See also* artificial players
 - `fireAll()`, 173
 - `fire(node)`, 172
 - `fireRandom()`, 173, 178
 - `fireSequence()`, 173
- discrete infinity, explained, 293
- discrete mechanics, 9. *See also* mechanics
 - in *Civilization*, 29
 - innovating with, 11
 - interaction with, 10
 - in *Zelda* games, 36
- dominant strategies, countering, 128–130
- Donkey Kong*, vs. *Super Mario Bros.*, 9
- Doom*, internal economy, 59
- Dormans, Joris, 79
- drains
 - function of, 95–96
 - versus sources, 61–62
- dynamic engine pattern, 153, 162
 - applicability, 305
 - collaborations, 306
 - consequences, 306
 - elaborating elements in, 162–163
 - examples, 307
 - implementation, 306–307
 - intent, 305
 - lock-and-key mechanisms, 255–258
 - in *Lunar Colony*, 212
 - motivation, 305
 - participants, 306
 - related patterns, 307
 - The Settlers of Catan*, 264
 - in *SimWar*, 188
 - structure, 306
 - type, 305
- dynamic friction pattern
 - applicability, 316
 - in *Caesar III*, 202
 - collaborations, 316
 - consequences, 317
 - examples, 317–318

dynamic friction pattern
(*continued*)

- explained, 155
- implementation, 317
- intent, 316
- lock-and-key mechanisms, 255–256
- in *Monopoly*, 186
- motivation, 316
- participants, 316
- related patterns, 318
- structure, 316
- type, 316

E

- Eco, Umberto, 287, 294, 298–299
- economic functions
 - converters, 62
 - drains, 62
 - sources, 61
 - traders, 62
- economic shapes, 62–64
 - charts, 62–63
 - chess, 64–65
 - figures, 62–63
 - graphs, 63
 - relating to mechanics, 65–71
- economy, defined, 59. *See also* internal economy
- economy construction games
 - building blocks, 77
 - maps, 77
 - meta-economic structure, 77
- economy-building games
 - effectiveness, 197
 - examples of, 197
 - goals in, 197
- ecosystems
 - complexity of, 51
 - feedback loop, 51
 - predator vs. prey, 51
- edutainment, 274
- elaboration, 13–14
 - applying to *Machinations* diagrams, 164
 - for converter element, 164
 - design focus, 165
 - explained, 162
 - of Harvester game, 163
 - reversing, 164

- vs. simplification, 164–165
- using as design tool, 162
- The Elder Scrolls* series, 32, 135
- Elite*
 - producing progress in, 263
 - travel and trade in, 310
- Elrod, Corvus, 18
- emergence. *See also* complex systems
 - Caesar III* game of, 206
 - categorizing in complex systems, 56–57
 - Civilization* example, 28–30
 - complexity barrier, 37
 - complexity of, 26–27
 - data and process intensity, 25
 - design considerations, 47
 - establishing goals for, 222
 - experiencing, 46
 - game states, 27–28
 - gameplay, 27–28
 - gameplay as, 43–47
 - harnessing, 57
 - history of, 23–24
 - integration with progression, 39–41
 - mechanics of, 38
 - order vs. chaos, 45–47
 - preference for, 24
 - probability space, 38
 - progress in, 224
 - vs. progression, 24–25, 30–31, 37–38
 - vs. randomness, 126–130
 - replay value, 45
 - vs. scripting, 268
 - structure of, 37–38
 - terminology, 26
 - water-tap experiment, 46
- emergent phases, progression through, 269
- emergent progression. *See also* progress
 - and gameplay phases, 266–267
 - overview, 258–259
 - pacing in, 266
 - reliance of board games on, 259
 - variation in, 266
- emergent storytelling, 262
- emergent vs. periodic systems, 45–47
- end conditions

- elements, 223
- triggering, 112
- enemies, adding to create choice, 231
- energy-harvesting game, 128–130, 163
- engine-building pattern
 - applicability, 311
 - in *Caesar III*, 202
 - collaborations, 312
 - consequences, 312
 - examples, 313
 - implementation, 312–313
 - intent, 311
 - in *Lunar Colony*, 212
 - motivation, 311
 - participants, 312
 - related patterns, 313
 - The Settlers of Catan*, 264
 - structure of, 312
 - type, 311
- entities
 - compound, 61
 - in *Monopoly*, 61
 - simple, 61
- equilibrium
 - changing, 66
 - defined, 66
 - dynamic vs. nondynamic, 70
 - of negative feedback mechanism, 70
 - shape of, 66
- escalating challenge pattern
 - applicability, 325
 - collaborations, 326
 - consequences, 326
 - examples, 232, 326
 - implementation, 326
 - intent, 325
 - motivation, 325
 - participants, 326
 - related patterns, 326
 - structure, 325
 - type, 325
- escalating complexity pattern, 232
 - applicability, 327
 - collaborations, 327
 - consequences, 328
 - examples, 328–329
 - in gameplay phases, 269
 - implementation, 328

- intent, 327
- motivation, 327
- participants, 327
- related patterns, 330
- structure, 327
- type, 327
- ethics and games, 282
- Experts Exchange online database, 275
- exponential curves, creating, 66–67

F

- feature freeze, 13
- feedback
 - basing on relative scores, 70–71
 - constructive vs. destructive, 322
- feedback basketball, 116–118
- feedback characteristics
 - durability, 122–124
 - effect, 122
 - investment, 122–123
 - range, 122–123
 - return, 122–123
 - speed, 122–123
 - type, 122, 124
- feedback loops
 - affecting outputs, 115
 - cards and armies, 119
 - closed circuits, 114
 - closing, 115
 - determining effects of, 123
 - ideal number of, 118
 - major vs. minor, 118
 - Monopoly*, 147
 - negative, 52
 - positive, 53
 - in *Risk*, 118
 - Risk*, 147
 - role in complex systems, 51–53
 - strength, 123
- feedback structures, 113. *See also*
 - determinability
 - affecting outputs, 115
 - closed circuits, 114–115
 - level of detail, 121
 - loops, 118–120
 - profiles, 121
- fighting mechanism, example of, 141
- films and games, 278–279
- `fireAll()` direct command, explained, 173
- `fire(node)` direct command, explained, 172
- `fireRandom()` direct command, explained, 173, 178
- `fireSequence()` direct command, explained, 173
- flower-collecting game, 231–236
- Foldit* crowdsourced search, 275
- Forest Temple
 - graph of mission, 34
 - map of, 34
- “Formal Abstract Design Tools,” 149
- formal methods, criticisms of, 166–167
- fortunes of players, charting, 63–64
- FPS economy, 136–138
 - ammunition, 137
 - enemies, 137
 - Engage* drains, 137
 - Kill* drains, 137
 - player health, 137
 - positive feedback loops, 137–138
- fractions game, *Refraction*, 274
- Frasca, Gonzalo, 296
- Fromm, Jochen, 56–57
- fun, relationship to learning, 271
- Fundamentals of Game Design*, 59, 169
- “The Future of Game Design,” 44

G

- Gabler, Kyle, 15
- gains vs. investments, 68–69
- Gamasutra*
 - “The Case for Game Design Patterns,” 151
 - “Formal Abstract Design Tools,” 149
 - forum, 150
- “Game Design as Narrative Architecture,” 32
- game design methodology, arguments against, 166–167
- Game Design Patterns*, 150
- game design process
 - concept stage, 13
 - documentation, 14
 - elaboration stage, 13–14
 - mechanics, 12–14
 - tuning stage, 13–14
- game design tools, arguments against, 166–167
- game economy, considering, 20
- game engines, open-source, 16–17
- game genres, 8
- Game Innovation Database, 150
- game mechanics. *See* mechanics
- Game of Goose* racing game, 133
- Game of Life, 53
 - cell states, 53
 - flocking birds, 54–55
 - glider, 54
 - grid structure, 53
 - iterations, 54
 - multiple emergence, 56
 - scales of organization, 56
 - starting, 53
- Game Ontology Project, 150
- game spaces
 - defined, 230
 - mapping mechanics to, 235–237
 - representing linearly, 235
 - reusing, 230
 - separating from missions, 230
- game states
 - changes in, 241
 - clarity of, 241
 - and gameplay, 27
 - possibilities of, 27
 - probability space, 27
 - trajectory, 27
- GameMaker* development environment, 17
- gameplay. *See also* play state
 - customizing via economy, 75–76
 - defined, 43
 - as emergence, 43–47
 - goal-oriented vs. free-form, 222
 - levels of, 226

gameplay (*continued*)
 martial arts principles,
 241–244

paidia vs. *ludus*, 222
 skill atoms, 238–240
 structuring, 221–223

gameplay phases
 charting in RTS game, 265–266
 composing, 268–269
 escalating complexity, 269
 initiating shifts between, 268
 multiple feedback, 269
 slow cycle, 268
 static engines, 268
 static friction, 268
 stopping mechanism, 268

games. *See also* reference games;

serious games
 balancing, 193
 of chance, 2
 defined, 1
 of emergence, 222
 ethics, 282
 films, 278–279
 hidden information in, 241
 hybrid example, 5
 mechanics of, 39
 and simulations, 284–288
 simulations in, 285–286
 as state machines, 2, 26
 unique quality of, 278
 unpredictability of, 2–3
 victory condition, 221–222

games of emergence. *See*
 emergence

games of progression. *See*
 progression

gamification, 275
 gaming vs. playing, 222

Gamma, Erich, 149
 “Gang of Four,” 149

gate types, 94, 302

gates

activation modes, 93
 automatic, 93
 conditional outputs, 93–94
 deterministic, 93–94
 distribution modes, 94
 interactive, 93
 output state connections, 95
 vs. pools, 93

probable outputs, 93–94
 random, 93–94
 types of, 93

Global Game Jam, 15
 goals of games, considering, 68

Grand Theft Auto

emergence and progression, 39
 progress in, 223
 reuse of game space in, 230
San Andreas, 25, 298–299

Grand Theft Auto III

debate about, 282
 intertextual irony, 298

graphs, using, 63

H

Half-Life series

action adventures, 25
 storytelling in, 32–33

Harvester game, elaborations of,
 128–130, 163

health, representing in games, 290

heater feedback mechanism,
 114–115

Helm, Richard, 149

hero's journey story pattern, 36

Historical Miniatures Gaming
 Society, 274

Holopainen, Jussi, 151

Hopson, John, 109

horizontal slice, creating for
 prototype, 16

hybrid game example, 5

hypotheses, testing, 284–285

I

icon, defined, 282–283

if statements

actions value, 175
 actionsOfCommand value, 175
 actionsPerStep value, 175
 pregen0...pregen9 value, 175
 random value, 175
 steps value, 175
 using with artificial players,
 173–174

improvisation, forcing, 126–127

index, defined, 282–283

intensity, data and process, 25

intentional emergence, explained,
 56

interactive nodes, drawing, 171

interactive stories, creating, 32

interface and control scheme,
 considering, 20

internal economy. *See also*
 economy

complementing physics, 71–72

converters, 62

customizing gameplay, 75–76

drains, 62

entities, 61

explained, 6

functions, 61–62, 83

game genres, 6

influencing progression, 72–73

probability spaces, 75–76

resources, 6, 60–61

sources, 61

traders, 62

intertextual irony, explained,
 298–299

intervals

dynamic, 109

vs. multipliers, 110

random flow rates, 109

using in Machinations
 diagrams, 108–109

inventory as analogous

simulation, 288–289

investments vs. gains, 68–69

J

Jakobson, Roman, 277

Jenkins, Henry, 32

Johann Sebastian Joust game, 5

Johnson, Ralph, 149

Juul, Jesper, 23–25

K

Kata martial arts principle,
 241–244

The Kids are Alright, 272

Kihon martial arts principle,
 241–244

Kihon-kata martial arts principle,
 241–244

Kings Quest, progress in, 223–224

Klondike, length of, 2
 Koster, Raph, 166, 271
 Kreimeier, Bernd, 150–151
Kriegsspiel game, 272–274
 Kumite martial arts principle, 241–244

L

The Landlord's Game, 272
 LARP (live-action role-play)
 session, 19
 law of diminishing returns, 156, 319
 learning
 martial arts principles, 241–244
 relationship to fun, 271
 LeBlanc, Marc, 70
The Legend of Zelda. *See* *Zelda* games
Leisure Suit Larry, progress in, 223–224
 less is more, 291–294
 levels of gameplay
 considering, 226
 designing, 229–231
 layout perspective, 229
 mission of, 230
 linear game space, representing, 235
 live-action role-play (LARP)
 session, 19
 lock-and-key mechanisms, 132
 abilities as keys, 251–252
 adding, 236–237
 cataloging mechanics, 255
 dynamic, 255–258
 dynamic friction, 255–256
 examples, 247
 explained, 247
 feedback mechanism, 255, 257
 machinations, 252–254
 missions and game spaces, 248–251
 player skill, 253
 vs. progress as resource, 260
The Longest Journey, 25
Lost Earth HD tower defense game, 50–51
 ludologists vs. narratologists, 31

ludus vs. *paidia*, 222
Lunar Colony
 actions gained, 219
 Actions register, 211
 balancing, 218
 building blocks, 213–216
 converter engine, 216
 described, 206–207
 design patterns, 212
 disadvantages, 216
 dynamic engine, 212
 economic strategies, 218–219
 economic structure, 211–212
 end conditions, 212
 engine building, 212
 events, 216–217
 game material, 207
 ice and ore lodes, 208
 ice mines, 209
 improving, 213
 levels for, 226–227
 obstacles, 216–217
 ore mines, 209
 playing, 209–210
 playing area, 207–208
 prototype, 207–211
 purifiers, 213–215
 raiding in, 218
 random events, 217
 refineries, 214–215
 removing dynamic engine, 216
 Resources pool, 211
 role of energy, 216
 rules, 207–210
 scripting scenarios, 217
 setup, 207–208
 stations, 209, 213–215
 stations as impediments, 217
 technology, 210
 transporters, 214–215
 way stations, 208–209
 winning, 210

M

Machinations diagrams. *See also*
 artificial players; mechanics;
 node types
 abstraction, 81–82
 action points, 88
 activation modes, 85

activators, 92
 adding artificial players to, 172
 analogous simulation, 292
 applying elaboration to, 164
 artificial player, 171
 asynchronous time mode, 87–88
 balancing, 195
 charts in, 114, 176–177
 color-coded, 97, 112–113
 colors in, 89
 connecting nodes, 91–92
 connections into nodes, 84
 delays, 110–111
 digital, 81
 end conditions, 97–98, 223
 engine categories, 153–154
 escalation categories, 157–158
 firing nodes automatically, 85
 friction categories, 155–156
 gates in, 93
 generating random numbers, 84
 goals in, 223
 hourglass example, 87
 input to node, 84
 interactive nodes, 85, 171
 interactive nodes in, 171
 intervals, 108–109
 label modifiers, 89–90
 level of detail, 81–82
 lock-and-key mechanisms, 252–255
 making calculations in, 107
 multiple feedback, 159
 multipliers, 109–110
 negative node resources, 90
 node modifiers, 90–91
 nodes, 82, 84–85
 origin of connection, 84
 output of node, 84
 passive nodes, 85, 91
 pattern descriptions, 151–152
 playing style reinforcement, 158
 pools, 83–85, 87, 94
 pulling resources, 85–86
 pushing resources, 85–86
 queues, 110–111
 random flow rates, 84
 registers, 107–108

Machinations diagrams (*continued*)
 resolving pulling conflicts, 88
 resource connections, 82–84, 87
 resources, 83
 reverse triggers, 111–112
Risk, 120
 scope, 81–82
 slow cycle, 160–161
 state changes, 89–92
 state connections, 82
 synchronous time mode, 87
 time modes, 87–88
 trade pattern, 159
 triggers, 91–92
 turn-based mode, 88
 worker placement, 160

Machinations framework
 design of, 82, 238
 explained, 57
 feedback structures, 80
 language syntax, 80
 overview, 80
 theoretical vision, 80

Machinations Tool
 bombling keys, 254
 features of, 81
 fighting mechanism, 141
 iterations, 81
 nondeterministic symbols, 125
 Quick Run mode, 84, 176–177
 resource connections, 84
 running, 81, 176–177
 time steps, 81
 using with internal economy, 83

Magic: The Gathering, 18, 126, 323

Magie, Elizabeth, 272

make the toy first, 15

Man, Play, and Games, 222

management simulation games, mechanics, 8

maps, using in economy construction, 77

Mario Galaxy, internal economy, 59

MarioKart, negative feedback mechanics in, 71

martial arts principles
 Kata, 241–244
 Kihon, 241–244

Kihon-kata, 241–244
 Kumite, 241–244

material number, producing, 64

mathematical strategists, 169

maze like structures, representing, 235

McLuhan, Marshall, 277–278

meaning
 appearance vs. mechanics, 296–298
 intertextual irony, 298–299
 layers of, 294–299
 unrelated, 295–296

mechanics. *See also* discrete mechanics; Machinations diagrams
 action games, 131–133
 core, 4
 designing, 14
 discrete vs. continuous, 9–12
 FPS economy, 136–138
 game design process, 12–14
 game genres, 7–8
 of games and stories, 39
 internal economy, 6
 limiting number of, 233
 mapping to game spaces, 235–237
 mapping to missions, 231–235
 versus mechanisms, 4
 media-independence, 4–6
 physics, 6, 9
 progression mechanisms, 6
 prototype development, 6
 racing games, 133–134
 randomizing in *Monopoly*, 182
 relating to economic shapes, 65–71
 RPG elements, 135–136
 RTS building, 139–140
 RTS fighting, 140–143
 RTS harvesting, 138–139
 versus rules, 3–4
 sending messages, 279–280
 social interaction, 7
 sources, 61
 structures, 30
 structures in, 226–228
 tactical maneuvering, 7
 technology trees, 143–144
 traders, 62

mechanisms, maximum number of, 292. *See also* progression mechanisms

mechanistic perspective, explained, 11

Meier, Sid, 28–30, 47

missions
 improving, 231
 mapping mechanics to, 231–235
 in open game spaces, 234
 separating from game spaces, 230

Monopoly
 artificial players, 179
 Available pool, 179
 buying houses, 184
 deterministic version, 180–181
 dynamic friction, 185–187
 effects of luck, 181–183
 entities in, 61
 feedback loops, 147
 feedback structures, 113
 mechanics of, 3–4
 model of, 179
 property tax mechanism, 186
 randomized rent mechanism, 182
 randomizing mechanics, 182–183
 removing randomness, 180
 rent and income balance, 183–185
 vs. *Risk*, 118
 rules of, 3–4
 as serious game, 272
 simulated play-test analysis, 180–181
 static friction, 315
 trend in game play, 180
 trigger in, 91–92
 two-player version, 179

multiple emergence, explained, 56

multiple feedback pattern, 202, 336

multipliers
 dynamic, 110
 vs. intervals, 110
 using in Machinations diagrams, 109–110

N

- narrative architecture, explained, 32
- narratologists vs. ludologists, 31
- negative feedback
 - basketball, 70, 116–117
 - creating equilibrium with, 65–66
 - effect of, 66
 - equilibrium, 70
 - explained, 52
 - incorporating, 203–204
 - rubberbanding, 71
- A New Kind of Science*, 49–50
- Nimitz, Chester, 274
- node types. *See also* Machinations diagrams
 - converters, 96
 - drains, 95–96
 - end conditions, 97–98
 - gates, 93–95
 - sources, 95
 - traders, 97
- nodes
 - activation modes, 85, 302
 - gate types, 302
 - pull and push modes, 85–86, 302
- nominal emergence, explained, 56

O

- order vs. chaos
 - emergent systems, 45–47
 - periodic systems, 45–46
- orthogonal unit differentiation, 142

P

- Pac-Man*
 - capture, 101–102
 - dots, 99–100, 103
 - fruit mechanism, 100–101, 103
 - ghost house, 101, 103
 - ghosts in, 55
 - loss of life, 101–102
 - Machinations diagram, 102–103

- modeling, 98–103
- power pills, 102–103
- resources, 98–99
- Threat* pool, 101, 103
- paidia* vs. *ludus*, 222
- paper prototyping, 17–19
 - advantages, 18
 - changing rules, 18–19
 - disadvantages, 19
 - LARP session, 19
- pattern descriptions. *See also* design patterns
 - Applicability, 152
 - Collaborations, 152
 - Consequences, 152
 - Examples, 152
 - Implementation, 152
 - Intent, 152
 - Motivation, 152
 - Name, 152
 - Participants, 152
 - Related Patterns, 152
 - Structure, 152
- A Pattern Language*, 148
- pattern language. *See also* design patterns
 - defined, 148
 - extending, 165
 - organization of, 149
- patterns, elaboration and nesting, 161–164
- paused state, explained, 2
- PeaceMaker*
 - design challenges, 281
 - mechanics sending messages, 279–280
- percentages
 - creating random values with, 84
 - representing probabilities as, 94
- periodic vs. emergent systems, 45–47
- perspectives, shifting, 224
- phase transitions, complexity theory applied to, 267
- physical mechanics, mixing with strategy, 10–11
- physical prototyping, 19
- physics
 - cartoon, 6

- complementing via economy, 71–72
- explained, 6
- game genres, 6
- mechanics of, 9
- use of, 71–72
- play spaces, learning from, 222
- play state, explained, 2. *See also* gameplay
- player skill, in lock-and-key mechanisms, 253
- player-centric design, 169
- players, measuring progress of, 225–226. *See also* artificial player
- playground, significance of, 222
- playing style reinforcement pattern. *See also* RPG elements
 - applicability, 333
 - collaborations, 334
 - consequences, 334
 - examples, 335–336
 - implementation, 335
 - intent, 333
 - motivation, 333
 - participants, 334
 - related patterns, 336
 - structure, 333
 - type, 333
- playing vs. gaming, 222
- poetic function, explained, 277
- Poole, Steven, 44
- pools
 - vs. gates, 93
 - vs. registers, 107–108
- positive feedback
 - amplifying differences, 68
 - basketball, 70–71, 116–117
 - deadlocks, 67
 - on destructive mechanisms, 68
 - effect of, 66–68
 - explained, 53
 - exponential curves, 66–67
 - mutual dependencies, 67
- Power Grid* board game, 169, 259–260
- production mechanism, 311
- random factors, 127–128
- stopping mechanism, 321

power-ups
 indicating, 131–132
 limited duration, 132

probability space
 creating via economy, 75–76
 explained, 26
 explosion of, 37
 shape of, 38

process intensity, 25

processes
 deterministic, 2, 129
 stochastic, 2

progress. *See also* emergent progression
 as aspect of game state, 259
 as character growth, 225
 as distance to target, 224–225
 vs. dynamic locks and keys, 260
 interaction with difficulty, 232
 as journey, 259, 261–262
 measuring, 260
 as player growth, 225–226
 producing indirectly, 262–265
 as resource, 260
 structuring, 223–226
 through completing tasks, 223–224
 complexity barrier, 37
 data and process intensity, 25
 designing, 36
 vs. emergence, 24–25, 30–31, 37–38
 goals in, 223
 history of, 23–24
 influencing via economy, 72–73
 integration with emergence, 39–41
The Legend of Zelda, 33–36
 mechanics of, 31
 ordered systems, 47
 structure of, 37–38
 through emergent phases, 269
 tutorials, 31

progression mechanisms. *See also* mechanisms
 explained, 6
 game genres, 6
 prototypes
 high-fidelity, 15

horizontal slice, 16
 low-fidelity, 16
 vertical slice, 16

prototyping process, speeding, 16–17

prototyping techniques
 focus, 19–21
 game economy, 20
 interface and control scheme, 20
 paper, 17–19
 physical, 19
 reference games, 21
 software, 16–17
 tech demos, 20
 tutorials, 21

Puerto Rico board game, 128

pull and push modes for nodes, 302

puzzle games, mechanics, 8

Q

“The Quest in a Generated World,” 35

queues, using in Machinations diagrams, 110–111

Quick Run option, using with Machinations Tool, 176–177

Quick Time Events, 292

R

race of accumulation, 69

racing games, rubber banding, 133–134

railroading, defined, 32

Rand, Ayn, 295

random flow rates
 multiplying, 109–110
 notations for, 84
 using with intervals, 109

random intervals, 109

random number generators, use of, 84

random values, creating, 84

randomness
 countering dominant strategies, 128–130
 vs. emergence, 126–130

forcing improvisation, 126–127

frequency, 126

impact, 126

realism vs. consistency, 44

reference games, picking, 21. *See also* games

Refraction fractions game, 274

registers
 interactive, 107–108
 passive, 107–108
 vs. pools, 107–108
 using in Machinations diagrams, 107–108

resource connections, label types, 301

resources
 abstract, 60–61
 concrete, 60–61
 consuming, 110–111
 converters, 62
 defined, 60
 happiness, 61
 intangible, 60
 producing, 110–111
 production rate, 61
 redistribution of, 91
 reputation, 61
 tangible, 60
 trading, 110–111
 using converters with, 96

reverse triggers, using in Machinations diagrams, 111–112

reward system, creating, *Super Mario Bros.*, 72

Risk, 24
 activating feedback loops, 119–120
 armies resource, 118
 capturing continents, 120
 core feedback loop, 118
 feedback loop, 147
 feedback profiles, 121
 gaining territories in, 119–120
 internal economy, 59
 level of detail, 121
 loss of territories, 120
 Machinations diagrams, 120
 vs. *Monopoly*, 118
 positive feedback loops, 121

- territories resource, 118
- rocket jumping, 44
- rock-paper-scissors,
 - unpredictability of, 3
- role-playing games, mechanics, 8
- roshambo/rochambeau of, 3
- RPG elements. *See also* playing
 - style reinforcement
 - experience points, 135
 - levels, 135
 - negative feedback, 136
 - positive feedback, 135
 - progress as character growth, 225
- RTS building, 139–140
- RTS fighting, 140–142
 - defensive mode, 142–143
 - offensive mode, 142–143
 - orthogonal unit
 - differentiation, 142
- RTS games
 - charting phases in, 267
 - turtling vs. rushing in, 188
- RTS harvesting, 138–139
- rubber banding, using in racing
 - games, 133–134
- rubberbanding, explained, 71
- rules
 - complexity of, 3, 45
 - for *Connect Four*, 27
 - function of, 1
 - impact on predictability, 3
 - versus mechanics, 3–4
 - for tic-tac-toe, 27
- rushing strategy, example in
 - SimWar, 192
- rushing vs. turtling, 188
- Ryan, Andrew, 295

S

- de Saint-Exupéry, Antoine, 292
- Sanders Peirce, Charles, 283
- de Saussure, Ferdinand, 283
- science, simulations in, 284–285
- science of complexity, 43. *See also*
 - complex systems
- scripting vs. emergence, 268
- SCV (Space Construction Vehicle), 67–69
- semiotics
 - defined, 282
 - development of, 284
 - games and simulations, 284–288
 - icons, 282–283
 - indexes, 282–283
 - signifier and signified, 283
 - symbols, 282–283
 - terminology, 283
 - as “the theory of the lie,” 287
- September 12*, 296–297
- serious games. *See also* games
 - Kriegsspiel*, 272–274
 - The Landlord's Game*, 272–273
 - Monopoly*, 272
 - simulation in, 288
 - war games, 272
- The Settlers of Catan*, 259–260, 263–264
 - dynamic engine, 264
 - economy of, 264–265
 - engine building pattern, 264
 - objective of, 263
- The Seven Cities of Gold*, 269
- Shakespeare, appeal of, 294–295
- Shannon, C. E., 27
- shapes. *See* economic shapes
- signifier and signified, defined, 283
- SimCity*, 23
 - disaster scenarios, 77
 - economies in, 197–198
 - economy construction, 76
 - mechanics sending messages, 279–280
 - meta-economic structure, 77
 - random maps in, 126
 - walkthrough for map, 25
- The Sims*
 - materialistic approach of, 265
 - measuring progress in, 265
- simulations
 - abstraction, 286–287
 - analogous, 288–289, 291–293
 - errors in, 287
 - in games, 285–286
 - in science, 284–285
 - in serious games, 288
 - symbolic, 290–293
- SimWar
 - artificial players, 191–192
 - attacking and defending, 189
 - average times, 193–194
 - building units, 189
 - color-coded resources, 189
 - combat, 189
 - costs of factories and units, 193–194
 - defensive units, 189
 - described, 187
 - draws or timeouts, 193–194
 - dynamic engine, 188
 - factories and resources, 188–189
 - modeling, 188–189
 - offensive units, 189
 - playing, 191
 - production costs, 192–193
 - random turtle player, 191
 - Resources pool, 188
 - rush wins, 193–194
 - rushing strategy, 192
 - spending resources, 189
 - strength of players, 190
 - turtle wins, 193–194
 - tweaking values, 192–194
 - two-player version, 190
 - visual summary, 187–188
- skill atoms
 - action event, 238–239
 - feedback event, 238–239
 - modeling event, 238–239
 - simulation event, 238–239
 - in *Super Mario Bros.*, 238–239
- skill of player, considering, 125
- skill trees, characteristics of, 238–239
- skills, learning vs. mastering, 240
- slow cycle pattern, 336
- Smith, Harvey
 - “The Future of Fame Design,” 44
 - orthogonal unit
 - differentiation, 142
- social games, mechanics, 8
- social interaction
 - explained, 7
 - game genres, 7
- software prototyping, 16–17
 - advantage, 17

- software prototyping (*continued*)
 - customization, 17
 - Spore*, 17
 - sources
 - versus drains, 61–62
 - explained, 61
 - representing for nodes, 95
 - space, depiction of, 32
 - Space Construction Vehicle (SCV), 67–69
 - Space Hulk*, asymmetrical attrition in, 324
 - Space Invaders*
 - trading progress points in, 260
 - victory conditions in, 221–222
 - spatial storytelling, 32
 - speed vs. cognitive effort, 232
 - Spore*, prototypes for, 17, 19
 - sports games, mechanics, 8
 - stability, creating in dynamic systems, 65–66
 - Star Wars: X-Wing Alliance*, 304
 - StarCraft*, 23
 - vs. *Civilization*, 40
 - comparing versions of, 227
 - “The Evacuation” mission, 40
 - harvesting minerals in, 307
 - harvesting raw materials in, 66–67, 69
 - narrative, 40
 - player performance, 69
 - resource distribution, 69
 - to *StarCraft 2*, 40–41
 - StarCraft II*
 - economy of, 226
 - “Outbreak” level, 227–228
 - resource harvesting in, 236–237
 - state connections
 - activators, 302
 - function of, 141
 - label modifiers, 301
 - label types, 302
 - node modifiers, 302
 - state machines
 - games as, 2, 26
 - probability space, 26
 - states, shifts between, 267
 - static engine
 - applicability, 303
 - collaborations, 303
 - consequences, 303–304
 - examples, 304–305
 - implementation, 304
 - intent, 303
 - motivation, 303
 - related patterns, 305
 - type, 303
 - static friction pattern
 - applicability, 314
 - collaborations, 314
 - consequences, 314
 - examples, 315
 - implementation, 314–315
 - intent, 314
 - motivation, 314
 - participants, 314
 - related patterns, 315
 - structure, 314
 - type, 314
 - stochastic processes, explained, 2
 - stopping mechanism pattern
 - applicability, 319
 - collaborations, 319
 - consequences, 320
 - examples, 320–321
 - implementation, 320
 - intent, 319
 - motivation, 319
 - participants, 319
 - related patterns, 321
 - structure, 319
 - type, 319
 - using in gameplay phases, 269
 - stories, mechanics of, 39
 - storytelling in games, 32, 228–229
 - avoiding repetition, 261
 - connecting events in, 261
 - emergent, 262
 - focusing on characters in, 261
 - ludologists, 31
 - narratologists, 31
 - progress as journey, 261–262
 - railroading, 32
 - StarCraft*, 40
 - strategic advantage, measuring, 64
 - strategy games
 - adding research to, 143
 - mechanics, 8
 - physical mechanics of, 10–11
 - tactical maneuvering in, 7
 - strong emergence, explained, 57
 - structural qualities
 - considering, 57
 - Machinations framework, 57
 - structures, defined, 30
 - subject-matter expert, working with, 288
 - subtasks
 - adding, 233–234
 - dependencies among, 234
 - Super Crate Box*, 329
 - Super Mario Bros.*, 131
 - board game for, 9
 - defeating enemies in, 291
 - vs. *Donkey Kong*, 9
 - fighting in, 291
 - Kata stage, 242
 - Kihon stage, 241–244
 - Kihon-kata stage, 241
 - Kumite stage, 242
 - reward system, 72
 - skill atom in, 238–239
 - skill tree, 239–240
 - symbolic simulation, 290–293
 - symbols, significance of, 282–283
- ## T
- tactical maneuvering
 - explained, 7
 - game genres, 7
 - “tank rush,” explained, 69
 - tasks
 - mutually exclusive, 235
 - optional, 235
 - tech demos, features of, 20
 - technology trees, modeling, 143
 - Tetris*, 26
 - escalating complexity, 329
 - feedback loop, 125
 - progression through emergent phases, 269
 - tetriminos in, 44
 - A Theory of Fun for Game Design*, 271
 - tic-tac-toe vs. *Connect Four*, 27
 - timed effect, creating, 111
 - tower defense games
 - activity level, 50
 - asymmetrical arms race, 332
 - components, 50
 - connections, 50
 - dynamic friction, 317

Lost Earth HD, 50–51
 trade pattern, 336
 traders
 vs. converters, 97
 explained, 62
Train, 297
 trajectory, role in game state, 27
Trigger Happy, 44
 tuning stage, 13–14
 turtling vs. rushing, 188
 tutorials
 building, 21
 creating, 31

U

Unity development environment, 17
 unpredictability, sources of, 3

V

vehicle simulation games,
 mechanics, 8
 vertical slice, creating for
 prototype, 16
 victory conditions, explained,
 221–222
 video games
 core mechanics, 4
 serious category of, 274–275
 views, shifting, 224
 Vlissides, John, 149
 Vogler, Christopher, 36

W

Wade, Mitchell, 272
 war games, history of, 272, 274
Warcraft
 converters, 62
 intangible resources, 60
 tangible resources, 60
WarCraft III, stopping mechanism
 in, 320
 Wardrip-Fruin, Noah, 39
 weak emergence, explained, 56
 weather system example, 47
 Wolfram, Stephen, 48–50, 53
 worker placement pattern, 336

World of Goo, 10–11, 26
 Wright, Will, 187

X

X-COM: UFO Defense, 38

Z

Zelda games, 25, 33–36
 bow and arrow, 254
 challenge to adventure, 36
 combat in, 35
 combining keys in, 254
 deadlock resolution in, 73
 discrete mechanics, 36
 dungeons, 35
 emergence and progression, 35
 Forest Temple level, 33–36
 gale boomerang, 35
 hub-and-spoke layout, 35
 keys consumed upon use, 253
 Link's adventures, 36
 lock and key mechanisms,
 35–36
 pottery as source, 73
 storytelling in, 228–229
Twilight Princess, 33, 36,
 243–244